

# A Mathematical Note on `dm_cmb.ipynb`: Patch-Based CMB Prior Learning with Flow Matching

Technical note for the diffusion-model CMB notebook

June 14, 2026

## Abstract

We document the mathematical content and implementation choices of the notebook `dm_cmb.ipynb`, which learns generative priors for  $(T, Q, U)$  patches of the cosmic microwave background (CMB) and Galactic foregrounds at 143 GHz and validates them through flat-sky spectral diagnostics. The workflow combines standard CMB harmonic methods—spin-weighted polarisation decompositions, full-sky simulations from CAMB power spectra, and curved-sky non-Gaussian lensing of the CMB primary by an independently sampled lensing potential through the `lenspyx/DUCC` remapping engine [1–4, 11, 12, 21]—with a modern path-based generative model trained by direct flow matching [8–10]. We describe in detail the pipeline used to bring Planck NPIPE / PR4 half-ring data and simulated priors to a common, well-defined representation: HEALPix pixelisation, harmonic downgrading, beam and pixel-window matching, sky masking, gnomonic patch extraction, spin-2 polarisation rotation, per-channel standardisation, and flat-sky spectrum estimation. The diffusion model is then specified completely: the Gaussian conditional probability path, the choice of signal/noise schedules, the U-Net vector-field architecture, the auxiliary radial Fourier-power regulariser used at training time, the deterministic Heun ODE sampler, and the spectral-validation diagnostics. The eventual scientific goal is posterior sampling for CMB component separation; this note focuses on the prior-learning stage, which is a prerequisite for any likelihood-based inference and which fixes the experimental conventions used in the rest of the work.

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>3</b>  |
| <b>2</b> | <b>Idealized CMB Power Spectra</b>                               | <b>4</b>  |
| <b>3</b> | <b>Realistic CMB Power Spectra</b>                               | <b>6</b>  |
| 3.1      | HEALPix pixelisation . . . . .                                   | 6         |
| 3.2      | Band-limit and downgrading . . . . .                             | 6         |
| 3.3      | Beam . . . . .                                                   | 7         |
| 3.4      | Pixel window . . . . .                                           | 8         |
| 3.5      | Masking . . . . .                                                | 8         |
| 3.6      | Flat-sky projection and patch spectra . . . . .                  | 9         |
| 3.7      | Patch-level standardisation . . . . .                            | 10        |
| 3.8      | Sum–difference noise estimates from independent splits . . . . . | 10        |
| <b>4</b> | <b>Data</b>                                                      | <b>10</b> |
| 4.1      | Observed CMB data . . . . .                                      | 12        |
| 4.2      | $p(f)$ : simulated CMB priors . . . . .                          | 13        |
| 4.3      | $p(x)$ : simulated foreground priors . . . . .                   | 16        |
| 4.4      | Conversion to flat patches and saved prior datasets . . . . .    | 17        |

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Deterministic flow matching for patch-based CMB priors</b>                       | <b>19</b> |
| 5.1      | Mathematical formalism: probability paths and flow matching                         | 19        |
| 5.1.1    | Probability paths and flow models                                                   | 19        |
| 5.1.2    | Conditional probability path and the marginalisation theorem                        | 20        |
| 5.1.3    | The conditional flow-matching loss                                                  | 21        |
| 5.1.4    | Score-velocity correspondence and the deterministic limit                           | 21        |
| 5.1.5    | Why deterministic flow matching for this work                                       | 22        |
| 5.2      | Probability path: Gaussian interpolant with linear schedule                         | 22        |
| 5.2.1    | Conditional velocity field and noise reparametrisation                              | 22        |
| 5.2.2    | Linear schedule                                                                     | 23        |
| 5.2.3    | Implications for training and sampling                                              | 24        |
| 5.3      | Network architecture                                                                | 24        |
| 5.3.1    | Why a U-Net for CMB patches                                                         | 24        |
| 5.3.2    | Macro-structure                                                                     | 25        |
| 5.3.3    | Time conditioning                                                                   | 25        |
| 5.3.4    | Parameter count                                                                     | 25        |
| 5.4      | Training: pure CFM loss and optimisation                                            | 25        |
| 5.4.1    | Loss function                                                                       | 25        |
| 5.4.2    | Symmetry-aware data augmentation                                                    | 26        |
| 5.4.3    | Time-sampling distribution                                                          | 26        |
| 5.4.4    | Optimiser                                                                           | 27        |
| 5.4.5    | Hyperparameter summary                                                              | 27        |
| 5.5      | EMA averaging and validation-driven checkpointing                                   | 27        |
| 5.5.1    | Exponential moving average of the weights                                           | 28        |
| 5.5.2    | The spectral selection score                                                        | 29        |
| 5.5.3    | Best-validation-spectral checkpointing                                              | 30        |
| 5.5.4    | End-to-end protocol summary                                                         | 31        |
| 5.6      | Sampling: deterministic ODE integration                                             | 31        |
| 5.6.1    | Numerical integrator: Heun’s second-order method                                    | 31        |
| 5.6.2    | Terminal-state reconstruction                                                       | 32        |
| 5.6.3    | Stability safeguards                                                                | 32        |
| 5.6.4    | Production sampling configuration                                                   | 32        |
| 5.7      | Diagnostics: validating the learned prior                                           | 32        |
| 5.7.1    | Reference ensembles                                                                 | 32        |
| 5.7.2    | Flat-sky $TT$ bandpower estimator                                                   | 33        |
| 5.7.3    | Bandpower binning conventions used in the notebook                                  | 33        |
| 5.7.4    | Pixel-space Gaussian smoothing ( <code>gaussian_smooth_patches.t</code> )           | 34        |
| 5.7.5    | Integrated and band-limited variance diagnostics                                    | 35        |
| 5.7.6    | Variance-matched comparison                                                         | 35        |
| 5.7.7    | Cosmic variance per bin                                                             | 36        |
| 5.7.8    | Why this is the relevant test for a CMB prior                                       | 36        |
| 5.7.9    | Reading the diagnostics: amplitude vs. shape biases                                 | 37        |
| <b>6</b> | <b>From the trained vector field to the <math>D_\ell^{TT}</math> plot</b>           | <b>37</b> |
| 6.1      | Step 1 – Initial noise in normalised patch units                                    | 37        |
| 6.2      | Step 2 – Heun integration of the flow ODE                                           | 38        |
| 6.3      | Step 3 – Endpoint denoising: $\mathbf{x}_{t_{\max}} \rightarrow \hat{\mathbf{x}}_0$ | 38        |
| 6.4      | Step 4 – De-standardisation to physical units                                       | 39        |
| 6.5      | Step 5 – Optional Gaussian pixel smoothing                                          | 39        |
| 6.6      | Step 6 – The estimator <code>flat_sky_tt_spectrum</code>                            | 39        |
| 6.7      | Step 7 – From $\hat{P}_b$ to $\hat{D}_b^{TT}$                                       | 41        |

|       |                                                                                   |           |
|-------|-----------------------------------------------------------------------------------|-----------|
| 6.8   | End-to-end summary . . . . .                                                      | 42        |
| 7     | <b>Foreground prior <math>p(x)</math>: training and post-training diagnostics</b> | <b>43</b> |
| 7.1   | Training configuration . . . . .                                                  | 44        |
| 7.2   | Per- $t$ velocity-field diagnostic . . . . .                                      | 44        |
| 7.3   | Post-training summary statistics: power spectrum and Minkowski functionals . .    | 45        |
| 7.3.1 | Flat-sky angular power spectrum . . . . .                                         | 45        |
| 7.3.2 | Minkowski functionals . . . . .                                                   | 45        |

# 1 Introduction

The cosmic microwave background (CMB) is a relic blackbody radiation field whose temperature and polarisation anisotropies encode the statistical properties of the very early universe and are central to modern cosmology [5, 12]. CMB observations are however contaminated by Galactic foregrounds (most notably thermal dust and synchrotron emission), instrumental noise, and the angular smoothing and sky masking imposed by realistic telescopes. Recovering the underlying CMB signal from the observed maps is therefore a component-separation problem.

In this work we expand on recent applications of deep generative modelling to CMB component separation [18] by exploring the use of *flow-matching* diffusion-style models [8–10] for joint inference of the foreground signal and the true CMB. Let  $y$  denote the observed map,  $x$  the foreground signal,  $f$  the *lensed* CMB,  $n$  the instrumental noise, and  $A$  the linear instrumental response (effective beam, pixel window, and sky mask). The observed signal model is

$$y = A(f + x) + n, \quad (1)$$

where the gravitational lensing of the CMB is treated implicitly through the prior  $p(f)$ : the simulated  $f$ -prior used for training is constructed by drawing unlensed Gaussian primary CMB realisations together with an independent realisation of the lensing potential  $\phi$  from a fiducial  $C_L^{\phi\phi}$  and then remapping the unlensed sky onto the deflected sphere with the `lenspyx` curved-sky lensing engine [21] (Sec. 4.2). The forward operator  $A$  therefore remains a linear post-lensing operator (effective beam, pixel window, and sky mask), while the non-Gaussianity induced by lensing on the CMB acoustic peaks, on the  $E \rightarrow B$  leakage and on the  $T/E \times \phi$  four-point function is carried by the simulated prior. Assuming a Gaussian noise model, the likelihood reads

$$p(y | x, f) \propto \exp\left[-\frac{1}{2} (y - A(f + x))^T N^{-1} (y - A(f + x))\right], \quad (2)$$

with  $N$  the noise covariance, and the posterior of interest factorises as

$$p(x, f | y) \propto p(y | x, f) p(x) p(f). \quad (3)$$

Component separation thus requires (i) a tractable likelihood, which is fixed once the instrumental pipeline and noise model are specified, and (ii) data-driven priors  $p(x)$  and  $p(f)$ , which we learn from realistic simulations. We train two independent U-Net models to parametrise vector fields  $u_t^\theta$  that generate probability paths from a simple Gaussian base distribution to each prior, and we sample those priors by integrating the corresponding ODE/SDE [10]. Posterior sampling is then obtained by augmenting the prior sampler with likelihood gradients  $\nabla_x \log p(y | x, f)$  and  $\nabla_f \log p(y | x, f)$  during integration; this is the only stage at which the observed CMB data  $y$  enters the inference.

We work at a single frequency, 143 GHz, using Planck NPIPE / PR4 half-ring maps as the observed sky  $y$  and the publicly released 143 GHz confidence mask as  $A$ 's sky cut [5]. Theoretical CMB spectra are computed with CAMB [4] for a fiducial  $\Lambda$ CDM cosmology and used, together with the lensing-potential auto-spectrum  $C_L^{\phi\phi}$ , to draw lensed-CMB  $f$ -priors that share the same

effective beam, pixel window, and sky footprint as the observations [11, 21]. Galactic foreground priors are obtained from PySM 3 [6, 7], which provides full-sky  $(I, Q, U)$  simulations of dust and synchrotron emission consistent with Planck data products; we use the small-scale stochastic presets `d10` and `s5`, so that successive foreground realisations are statistically independent draws of the small-scale Galactic field rather than rotated copies of a fixed template. All maps are processed in the HEALPix pixelisation [3, 13] and the diffusion model is trained on flat-sky  $64 \times 64$  patches obtained by gnomonic projection.

The scientific motivation is twofold. First, by learning  $p(f)$  and  $p(x)$  from data we obtain priors that capture non-Gaussian morphology that simple parametric models cannot represent, in particular for Galactic foregrounds. Second, the resulting posterior sampler outputs an ensemble of plausible component maps consistent with the data, providing both a point estimate of  $(x, f)$  and a calibrated uncertainty on it. This is a prerequisite for unbiased downstream cosmological inference, including the search for primordial  $B$ -modes for which residual foreground contamination is the dominant systematic.

## 2 Idealized CMB Power Spectra

We briefly review the spherical-harmonic description of CMB temperature and polarisation anisotropies, since this is the natural language in which our priors are constructed and validated. A more pedagogical exposition can be found in Refs. [3, 11, 12].

The CMB temperature anisotropy is a scalar field on the sphere and is therefore naturally expanded in spherical harmonics [1, 3, 12]:

$$T(\hat{n}) = \sum_{\ell=0}^{\infty} \sum_{m=-\ell}^{\ell} a_{\ell m}^T Y_{\ell m}(\hat{n}). \quad (4)$$

From the orthonormality of the spherical-harmonics

$$\int_{\mathbb{S}^2} Y_{\ell m}(\hat{n}) Y_{\ell' m'}^*(\hat{n}) d\Omega = \delta_{\ell\ell'} \delta_{mm'}, \quad (5)$$

the harmonic coefficients are obtained by

$$a_{\ell m}^T = \int_{\mathbb{S}^2} T(\hat{n}) Y_{\ell m}^*(\hat{n}) d\Omega. \quad (6)$$

The coefficient  $a_{\ell m}^T$  is therefore the amplitude with which the angular pattern  $Y_{\ell m}$  appears in the sky, and in a statistically isotropic theory the spherical-harmonic basis is preferred because two-point statistics become diagonal in  $(\ell, m)$ . The multipole  $\ell$  controls angular scale through the rough correspondence

$$\theta \sim \frac{\pi}{\ell}, \quad (7)$$

so small  $\ell$  corresponds to large coherent angular structures and large  $\ell$  to fine structure; the azimuthal index  $m$  resolves the angular dependence within a fixed  $\ell$ .

CMB polarisation is described by the Stokes parameters  $Q$  and  $U$  defined relative to a chosen local basis on the sphere:  $Q$  measures the linear polarisation aligned with the basis (up/down minus left/right), and  $U$  the same quantity rotated by  $45^\circ$ . Unlike  $T$ , the pair  $(Q, U)$  is not a scalar but transforms as a *spin*- $\pm 2$  field under local rotations of the basis, and its natural harmonic expansion uses spin-weighted spherical harmonics [1, 2, 11]:

$$(Q \pm iU)(\hat{n}) = \sum_{\ell m} a_{\ell m}^{\pm 2} Y_{\ell m}(\hat{n}), \quad (8)$$

with coefficients obtained by projection,

$$a_{\ell m}^{\pm 2} = \int_{\mathbb{S}^2} (Q \pm iU)(\hat{n}) {}_{\pm 2}Y_{\ell m}^*(\hat{n}) d\Omega. \quad (9)$$

The scalar  $E$ - and pseudo-scalar  $B$ -mode coefficients are defined by [1, 2]

$$a_{\ell m}^E = -\frac{1}{2} (a_{\ell m}^2 + a_{\ell m}^{-2}), \quad a_{\ell m}^B = \frac{i}{2} (a_{\ell m}^2 - a_{\ell m}^{-2}). \quad (10)$$

$E$  is the curl-free, parity-even component of the polarisation and  $B$  the divergence-free, parity-odd component. The decomposition is local in harmonic space but non-local in pixel space, so finite sky coverage induces  $E/B$  mode mixing in any flat-sky or masked estimator.

For a statistically isotropic Gaussian CMB, the harmonic coefficients satisfy

$$\mathbb{E}[a_{\ell m}^X (a_{\ell' m'}^Y)^*] = C_{\ell}^{XY} \delta_{\ell \ell'} \delta_{m m'}, \quad X, Y \in \{T, E, B\} \quad (11)$$

and all statistical information is contained in the covariance matrix at each  $\ell$ :

$$\Sigma_{\ell} = \begin{pmatrix} C_{\ell}^{TT} & C_{\ell}^{TE} & 0 \\ C_{\ell}^{TE} & C_{\ell}^{EE} & 0 \\ 0 & 0 & C_{\ell}^{BB} \end{pmatrix}. \quad (12)$$

We can compute two-point functions in terms of the  $C_{\ell}^{XY}$  coefficients. The full temperature two-point function for instance is:

$$\xi_T(\hat{n}, \hat{n}') := \mathbb{E}[T(\hat{n})T(\hat{n}')] \quad (13)$$

$$\begin{aligned} &= \mathbb{E} \left[ \sum_{\ell m} a_{\ell m}^T Y_{\ell m}(\hat{n}) \sum_{\ell' m'} (a_{\ell' m'}^T)^* Y_{\ell' m'}^*(\hat{n}') \right] \\ &= \sum_{\ell m} \sum_{\ell' m'} \mathbb{E}[a_{\ell m}^T (a_{\ell' m'}^T)^*] Y_{\ell m}(\hat{n}) Y_{\ell' m'}^*(\hat{n}') \\ &= \sum_{\ell=0}^{\infty} C_{\ell}^{TT} \sum_{m=-\ell}^{\ell} Y_{\ell m}(\hat{n}) Y_{\ell m}^*(\hat{n}') \\ &= \sum_{\ell=0}^{\infty} \frac{2\ell+1}{4\pi} C_{\ell}^{TT} P_{\ell}(\hat{n} \cdot \hat{n}'). \end{aligned} \quad (14)$$

where we used

$$\sum_{m=-\ell}^{\ell} Y_{\ell m}(\hat{n}) Y_{\ell m}^*(\hat{n}') = \frac{2\ell+1}{4\pi} P_{\ell}(\hat{n} \cdot \hat{n}'). \quad (15)$$

Instead of  $C_{\ell}^{XY}$  it is common to plot

$$D_{\ell}^{XY} \equiv \frac{\ell(\ell+1)}{2\pi} C_{\ell}^{XY}, \quad (16)$$

which is visually more informative because the prefactor  $\ell(\ell+1)$  roughly cancels the leading  $\ell^{-2}$  decline of an approximately scale-invariant  $C_{\ell}$  and makes the acoustic peaks easier to see [11].

In the large- $\ell$  limit,  $D_{\ell}^{TT}$  admits a clean physical interpretation as the temperature variance per logarithmic  $\ell$ -interval. Setting  $\hat{n}' = \hat{n}$  in the two-point function and using  $P_{\ell}(1) = 1$ , we have

$$\text{Var}[T(\hat{n})] = \sum_{\ell=0}^{\infty} \frac{2\ell+1}{4\pi} C_{\ell}^{TT}. \quad (17)$$

For  $\ell \gg 1$ , approximate the sum by an integral and use  $2\ell + 1 \simeq 2\ell$ :

$$\begin{aligned} \text{Var}[T(\hat{n})] &\simeq \int_0^\infty \frac{\ell C_\ell^{TT}}{2\pi} d\ell = \int_0^\infty \frac{\ell^2 C_\ell^{TT}}{2\pi} d \ln \ell \\ &\simeq \int_0^\infty \frac{\ell(\ell+1) C_\ell^{TT}}{2\pi} d \ln \ell = \int_0^\infty D_\ell^{TT} d \ln \ell. \end{aligned} \quad (18)$$

The same argument applies to  $D_\ell^{EE}$  and  $D_\ell^{BB}$ , so the four spectra  $\{D_\ell^{TT}, D_\ell^{EE}, D_\ell^{BB}, D_\ell^{TE}\}$  provide an interpretable, scale-resolved summary of the second-order statistics of the Gaussian CMB.

**Cosmic variance.** Even with a perfect noiseless full-sky map, the empirical estimator  $\hat{C}_\ell^{XY}$  has an irreducible variance [11, 12]

$$\text{Var}[\hat{C}_\ell^{XY}] = \frac{1}{2\ell+1} [C_\ell^{XX} C_\ell^{YY} + (C_\ell^{XY})^2], \quad (19)$$

which arises from the finite number  $(2\ell+1)$  of independent  $m$ -modes available at each multipole. This sets a fundamental floor for the precision with which any model, including the present one, can be tested at a given  $\ell$ , and is largest at low  $\ell$ . On a fraction  $f_{\text{sky}}$  of the sky, the variance is enlarged by approximately  $1/f_{\text{sky}}$ , defining the so-called *sample variance* regime relevant to masked analyses [11].

### 3 Realistic CMB Power Spectra

In practice the idealised description above must be modified to reflect (i) finite-resolution map representations, (ii) instrumental smoothing by the telescope beam, (iii) angular averaging into pixels, (iv) sky cuts that null contaminated regions, and (v) the geometry of patch-based analyses. We summarise here the formalism associated with each of these effects in general terms, independent of any particular data set or simulation; the concrete numerical choices, instrumental parameters, and software implementations adopted in this work are deferred to Sec. 4. The presentation closely follows the conventions of Refs. [3, 11, 12].

#### 3.1 HEALPix pixelisation

We adopt the HEALPix scheme [3, 13] as the all-sky pixelisation throughout. A HEALPix map discretises the sphere into pixels of *equal area* arranged on isolatitude rings, controlled by a single resolution parameter  $N_{\text{side}}$  (a power of two). The number of pixels and the pixel angular size are

$$N_{\text{pix}} = 12 N_{\text{side}}^2, \quad \theta_r = \sqrt{\frac{4\pi}{N_{\text{pix}}}} = \sqrt{\frac{\pi}{3 N_{\text{side}}^2}}, \quad (20)$$

where  $\theta_r$  is the side-length of a pixel in radians [3, 11]. Equal-area pixels permit the spherical-harmonic transform to be computed by a single ring-by-ring quadrature without per-pixel weighting, and allow second-order statistics to be averaged uniformly across the sky. HEALPix maps may be stored in either “RING” or “NESTED” ordering; the spherical-harmonic transforms used below assume the RING convention, which is the default for most CMB data products and software [3].

#### 3.2 Band-limit and downgrading

A continuous field on the sphere admits a spherical-harmonic expansion up to  $\ell_{\text{max}} = \infty$ . A HEALPix map is intrinsically discrete and therefore band-limited; following standard practice

one adopts

$$\ell_{\max} = 3 N_{\text{side}} - 1, \quad (21)$$

above which the spherical-harmonic transform is no longer reliable due to aliasing of unresolved structure into the represented modes [3, 11]. Ref. [11] also recommends the more conservative choice  $\ell_{\max} = 2 N_{\text{side}}$  when transitioning between resolutions, an inequality that is essentially saturated whenever the beam suppression at the upper end of the represented multipole range is severe.

**Harmonic-space downgrading.** Bringing a map from a high input resolution  $N_{\text{side}}^{\text{in}}$  to a lower target resolution  $N_{\text{side}}^{\text{out}}$  is conceptually a band-limit followed by a resampling. The recommended procedure [11] is to perform the downgrade entirely in harmonic space rather than through pixel-space averaging, because pixel-space routines such as `ud_grade` implicitly convolve with the new pixel window and, more importantly, treat  $Q$  and  $U$  as scalars, breaking the spin-2 transformation law of polarisation and producing biased  $E/B$  power. Concretely, one

1. computes polarised harmonic coefficients  $(a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B)$  of the input map up to

$$\ell_{\max}^{\text{in}} = \min(3 N_{\text{side}}^{\text{in}} - 1, \ell_{\max}^{\text{out}}), \quad (22)$$

i.e. explicitly nulls all multipoles above the target band-limit, as required when synthesising at the lower  $N_{\text{side}}$  [11];

2. rescales the harmonic coefficients to the desired output beam and pixel window,  $a_{\ell m}^{\text{out}} = (B_{\ell}^{\text{out}} P_{\ell}^{\text{out}}) / (B_{\ell}^{\text{in}} P_{\ell}^{\text{in}}) a_{\ell m}^{\text{in}}$  (the Sullivan–Hergt–Scott downgrading recipe [11]), where  $B_{\ell}$  is the beam transfer function of Sec. 3.3 and  $P_{\ell}$  is the HEALPix pixel-window function of Sec. 3.4;
3. synthesises the output  $(T, Q, U)$  maps at  $N_{\text{side}}^{\text{out}}$  using the rescaled coefficients.

When the input pixel window is sufficiently close to unity in the represented multipole range the second factor reduces to a pure beam rescaling,  $a_{\ell m}^{\text{out}} = (B_{\ell}^{\text{out}} / B_{\ell}^{\text{in}}) a_{\ell m}^{\text{in}}$ ; this approximation should be checked rather than assumed.

### 3.3 Beam

The optics of any real CMB telescope smear the underlying sky with a finite-width beam, which we model as a circular Gaussian of full-width-half-maximum  $\theta_{\text{FWHM}}$  and Gaussian width  $\sigma$  related by [11]

$$\sigma = \frac{\theta_{\text{FWHM}}}{\sqrt{8 \ln 2}}. \quad (23)$$

For an azimuthally symmetric beam, real-space convolution becomes multiplication in harmonic space, with the scalar transfer function

$$B_{\ell} = \exp\left[-\frac{1}{2} \ell(\ell+1) \sigma^2\right], \quad (24)$$

so that observed temperature spectra are damped at high  $\ell$  according to

$$C_{\ell}^{TT, \text{obs}} = B_{\ell}^2 C_{\ell}^{TT}. \quad (25)$$

This is the same  $B_{\ell}$  that appears in the harmonic downgrading formula of Sec. 3.2 and in the combined transfer-function model of Sec. 3.4 (Eq. 28). Polarisation, being a spin-2 field, picks up a small additional correction [11, 16]: the polarised beam is approximately  ${}_2B_{\ell} = B_{\ell} e^{2\sigma^2}$ . At sub-degree resolution this correction is exponentially suppressed and the scalar form (24) is

sufficient for both temperature and polarisation; we adopt this approximation in all subsequent expressions.

When data and simulations must be combined coherently, all maps should be brought to a common effective beam. If a map carries an input beam of FWHM  $\theta_{\text{in}}$  and the target resolution is  $\theta_{\text{out}} \geq \theta_{\text{in}}$ , this is achieved by multiplying its harmonic coefficients by the residual Gaussian

$$B_\ell^{\text{add}} = \exp\left[-\frac{1}{2} \ell(\ell+1) \sigma_{\text{add}}^2\right], \quad \sigma_{\text{add}} = \frac{1}{\sqrt{8 \ln 2}} \sqrt{\theta_{\text{out}}^2 - \theta_{\text{in}}^2}. \quad (26)$$

Enforcing  $\theta_{\text{out}} \geq \theta_{\text{in}}$  keeps the operation purely additional smoothing, with no deconvolution and therefore no noise amplification at high  $\ell$ . Bringing the data and the prior simulations to a single, well-defined effective resolution is essential for any likelihood-based component-separation analysis, where the residual  $y - A(f+x)$  of the forward model in Sec. 1 must live at a common resolution [11].

### 3.4 Pixel window

Beam smoothing accounts for the optical blur of the telescope; *pixelisation* introduces an additional, distinct angular smoothing that arises from averaging the sky over the area of each pixel [11]. Working at fixed  $N_{\text{side}}$ , the corresponding harmonic transfer function  $P_\ell^{N_{\text{side}}}$  is provided by HEALPix [3] and is well approximated at moderate  $\ell$  by a sinc,

$$P_\ell^{\text{sinc}} = \text{sinc}\left(\frac{\ell \theta_r}{2\pi}\right), \quad (27)$$

where  $\theta_r$  is the pixel size from Eq. (20). The full HEALPix function is computed by spherical-harmonic transform of an average pixel for  $N_{\text{side}} \leq 128$  and extrapolated by a top-hat approximation at larger  $N_{\text{side}}$  [3]. Combining beam and pixel-window contributions, the model for an observed power spectrum becomes

$$C_\ell^{\text{obs}} = P_\ell^2 B_\ell^2 C_\ell^{\text{true}}. \quad (28)$$

Here  $B_\ell$  is the Gaussian beam transfer function of Sec. 3.3 (Eq. 24) and  $P_\ell \equiv P_\ell^{N_{\text{side}}}$  is the HEALPix pixel-window function defined above; both operators also appear explicitly in the harmonic downgrading formula of Sec. 3.2. HEALPix [3] provides separate pixel windows  $P_\ell^T$  and  $P_\ell^P$  for scalar and spin-2 fields, and a careful analysis applies the appropriate one to  $a_{\ell m}^T$  and to  $\{a_{\ell m}^E, a_{\ell m}^B\}$  respectively. The relative magnitudes of  $B_\ell$  and  $P_\ell$  depend on the specific beam and resolution chosen; in regimes where the beam smoothing dominates, the pixel-window correction can be subdominant but is rarely negligible at the highest represented multipoles [11], and should be evaluated quantitatively before being neglected.

### 3.5 Masking

The Galactic plane and bright extragalactic point sources contaminate the CMB at all frequencies, and any analysis on real data must restrict itself to a sufficiently clean fraction of the sky [5, 11]. This is encoded by a sky mask  $M(\hat{n}) \in [0, 1]$ , typically a HEALPix map at the resolution of the data, where  $M = 1$  flags pixels retained by the analysis and  $M < 1$  flags partially or fully contaminated regions. When the mask is provided at higher resolution than the analysis pixelisation, it is downgraded with a scalar pixel-space averaging (`ud_grade`) – legitimate here because  $M$  is a scalar weighting and not a spin field – and typically thresholded above some fraction  $M_{\text{min}} \in [0, 1]$  to enforce a deterministic clean-sky/contaminated dichotomy.

The choice of how to combine the mask with the power-spectrum estimator is non-trivial. A naive multiplication  $\tilde{T} = M T$  followed by a full-sky harmonic transform biases all spectra

by approximately a factor of the unmasked sky fraction

$$f_{\text{sky}} = \frac{1}{N_{\text{pix}}} \sum_{i=1}^{N_{\text{pix}}} M_i^2, \quad (29)$$

and additionally couples power across multipoles, leading to ringing and large biases especially for  $EE/BB$  where the much weaker polarisation signal sits underneath leakage from the masked-out regions. State-of-the-art alternatives include the pseudo- $C_\ell$  **MASTER** estimator [14] as implemented in **NaMaster** [15], and quasi-maximum-likelihood estimators for the lowest  $\ell$  [11].

A simpler alternative, applicable to patch-based analyses (Sec. 3.6), is to bypass pseudo- $C_\ell$  altogether by restricting attention to fully-clean patches: candidate patch *centres* are drawn from  $\{i : M_i \geq M_{\text{min}}\}$  and any candidate whose projected footprint contains a pixel with  $M < M_{\text{min}}$  is rejected. The retained ensemble is then effectively unmasked, and the flat-sky FFT-based spectral estimators of Sec. 3.6 are not biased by mode-coupling. The cost is a non-trivial reduction of usable sky and a small mask-induced bias in the patch ensemble's prior over sky position, both of which are easy to quantify directly. Apodising the mask before per-patch acceptance and computing pseudo- $C_\ell$  on each patch with **NaMaster** [15] would relax the acceptance criterion and recover usable sky in the partially apodised border regions, at the price of restoring a small mode-coupling correction.

### 3.6 Flat-sky projection and patch spectra

When a generative model or any other translation-invariant learner is trained on small angular regions rather than full-sky maps, the natural data representation is a flat two-dimensional patch. The angular size of a patch is small enough that curvature corrections are subdominant, while still being large enough to resolve the multipoles of interest, so spherical-harmonic operators can be replaced by translation-invariant 2D Fourier operators at the price of a controlled approximation error.

A patch is obtained by a *gnomonic* (tangent-plane) projection of the underlying HEALPix map onto a square grid of  $N_{\text{patch}} \times N_{\text{patch}}$  pixels at angular sampling  $\theta_{\text{patch}}$ , centred on a chosen sky direction  $\hat{n}_0$ , with linear extent

$$L = N_{\text{patch}} \theta_{\text{patch}}. \quad (30)$$

For  $L \lesssim 10^\circ$ , curvature corrections to the spherical metric are of order  $L^2 \sim \text{few} \times 10^{-2}$  and the flat-sky approximation is quantitatively accurate [11, 17]. The temperature channel is resampled by simple scalar interpolation (e.g. `healpy.projector.GnomonicProj`). For polarisation, however, a naive scalar resampling of  $Q$  and  $U$  is incorrect: under the change of basis induced by the projection,  $(Q, U)$  acquires a position-dependent rotation by an angle  $\psi(\hat{n})$ , and a *spin-2 rotation*

$$\begin{pmatrix} Q' \\ U' \end{pmatrix} = \begin{pmatrix} \cos 2\psi & \sin 2\psi \\ -\sin 2\psi & \cos 2\psi \end{pmatrix} \begin{pmatrix} Q \\ U \end{pmatrix} \quad (31)$$

must be applied at each pixel [2, 17]. The angle  $\psi(\hat{n})$  can be computed analytically or numerically by finite-differencing the projection along the local north direction. Without this rotation,  $E$  and  $B$  constructed from the patch FFT contain a spurious geometric mixing of the two parities that biases polarised spectra, especially in  $BB$  where the signal is intrinsically smaller.

On each patch one estimates spectra via the flat-sky power spectrum estimator [11, 17]: for an angular footprint of side  $L$  and pixel size  $\theta_{\text{patch}}$ , the discrete two-dimensional Fourier modes are

$$\ell = 2\pi \mathbf{k}, \quad \mathbf{k} = \left( \frac{n_x}{L}, \frac{n_y}{L} \right) \text{ rad}^{-1}, \quad n_x, n_y \in \mathbb{Z}, \quad (32)$$

and the spin-2 field decomposes into flat-sky  $E/B$  via

$$\tilde{E}(\ell) = -\tilde{Q}(\ell) \cos 2\phi_\ell - \tilde{U}(\ell) \sin 2\phi_\ell, \quad \tilde{B}(\ell) = \tilde{Q}(\ell) \sin 2\phi_\ell - \tilde{U}(\ell) \cos 2\phi_\ell, \quad (33)$$

where  $\phi_\ell$  is the angle of  $\ell$  in the patch frame [2, 17]. The estimator

$$\hat{C}_\ell^{XY} = \frac{\theta_{\text{patch}}^2}{N_{\text{patch}}^2} \langle \tilde{X}(\ell) \tilde{Y}^*(\ell) \rangle_{|\ell| \in \Delta\ell} \quad (34)$$

averages cross-products in annular bins of  $|\ell|$  [11], where the angular range is bounded from below by the fundamental mode  $\ell_{\min} \simeq 2\pi/L$  and from above by the Nyquist multipole  $\ell_{\max} \simeq \pi/\theta_{\text{patch}}$ . When patch centres are restricted to clean sky and full-patch validity is enforced (Sec. 3.5), no apodisation or pseudo- $C_\ell$  correction is required at the patch level; the mode-coupling matrix reduces to the flat-sky identity for an isolated, fully-unmasked patch, and the bias for  $f_{\text{sky}}^{\text{patch}} = 1$  is negligible.

### 3.7 Patch-level standardisation

Independently of the spherical-harmonic formalism, training a generative model requires that the input fields have well-conditioned numerical scales. Raw  $T$ ,  $Q$ ,  $U$  patches in  $\mu\text{K}_{\text{CMB}}$  have very different per-channel variances ( $\sigma_T \gg \sigma_{Q,U}$  at intermediate frequencies, where the temperature acoustic peaks dominate over the polarised signal), which would force a generative model to learn this trivial scale hierarchy at the cost of higher-order structure and can induce unstable training near schedule endpoints [8, 18]. A standard remedy is the deterministic per-channel standardisation

$$\tilde{X}_c(\mathbf{x}) = \frac{X_c(\mathbf{x}) - \mu_c}{\sigma_c}, \quad c \in \{T, Q, U\}, \quad (35)$$

where  $\mu_c$  and  $\sigma_c$  are computed once on the training split only and then applied verbatim to any held-out splits. The generator is trained on  $\tilde{X}$ ; at inference time samples are de-normalised by inverting this transformation before computing physical-units diagnostics. The procedure is lossless in the sense that a perfect generator of  $p(\tilde{X})$  yields a perfect generator of  $p(X)$ ; it does mean, however, that intermediate spectra plotted in normalised units are scale-rescaled versions of the physical  $C_\ell$ , and the two domains must be distinguished explicitly in any diagnostic.

### 3.8 Sum–difference noise estimates from independent splits

Modern CMB experiments commonly deliver pairs of independent map-level splits constructed from disjoint subsets of the time-ordered data, e.g. half-mission, half-ring, or odd/even-detector splits. Given two such splits  $h_1$  and  $h_2$  of an observed sky, one forms the standard sum and difference combinations

$$y = \frac{1}{2}(h_1 + h_2), \quad d = \frac{1}{2}(h_1 - h_2). \quad (36)$$

Under the assumption that the sky signal is identical in both halves and the noise is independent between them,  $y$  retains the full sky signal and approximately half the noise power, while  $d$  is a sky-signal-free realisation of the noise on the map. The map  $y$  is therefore the natural observable on which to base inference, while  $d$  provides a model-independent diagnostic of the instrumental noise: its empirical variance estimates the diagonal of the per-pixel noise covariance, its auto-spectrum estimates the noise power spectrum  $N_\ell$ , and its lack of cross-correlation with any clean simulation of the sky provides a sanity check on the split assumption.

## 4 Data

This section instantiates the general formalism of Sec. 3 on the three data products fed to the prior-learning pipeline: the observed Planck 143 GHz sky  $y$ , the simulated lensed-CMB prior  $p(f)$ , and the simulated Galactic-foreground prior  $p(x)$ . We work at a single observing frequency, 143 GHz, where the 143 GHz channel of Planck has favourable signal-to-noise on the CMB and where the Galactic-foreground contamination is dominated by a narrow set of

components (thermal dust and synchrotron) for which standard simulators exist. All maps are stored in HEALPix RING ordering and in  $\mu\text{K}_{\text{CMB}}$  throughout.

**Common processing parameters.** By construction all three products live in the same numerical and angular domain (Table 1 summarises the choices). We adopt:

- HEALPix resolution  $N_{\text{side}} = 512$ , corresponding to  $N_{\text{pix}} = 12N_{\text{side}}^2 = 3,145,728$  pixels and a pixel scale  $\theta_r \simeq 6.87'$  (Sec. 3.1). This is the lowest power-of-two  $N_{\text{side}}$  that comfortably band-limits maps smoothed to the target  $10'$  beam, while keeping the cost of training a U-Net on patches manageable.
- Harmonic cutoff  $\ell_{\text{max}} = 3N_{\text{side}} - 1 = 1535$  in all spherical-harmonic transforms (Sec. 3.2).
- Common effective beam: a circular Gaussian of FWHM  $\theta_{\text{out}} = 10'$  applied to all three products via Eq. (24) (Sec. 3.3). This common-beam choice is what makes the data and the prior simulations harmonically compatible.
- Pixel windows: the polarisation-aware HEALPix windows  $P_\ell^T$  and  $P_\ell^P$  at  $N_{\text{side}} = 512$  are applied uniformly to both simulated priors  $p(f)$  and  $p(x)$  via a common harmonic-space step (Sec. 3.4), so that  $p(f)$  and  $p(x)$  share an identical effective transfer function. At this configuration  $P_\ell^{T,P}$  contributes at most an extra  $\sim 8\%$  damping at  $\ell = \ell_{\text{max}}$  relative to the beam alone. Pixel-window deconvolution is not applied to the observed maps; the corresponding mismatch is bounded by the input pixel-window correction at  $N_{\text{side}}^{\text{in}} = 2048$ ,  $\lesssim 0.3\%$  over the represented multipole range.
- Common sky mask: the public Planck 143 GHz confidence mask  $M(\hat{n})$ , downgraded to  $N_{\text{side}} = 512$  with `healpy.ud_grade` and thresholded at  $M_{\text{min}} = 0.9$  to define the clean-sky footprint (Sec. 3.5).
- Patch geometry: square  $N_{\text{patch}} \times N_{\text{patch}} = 64 \times 64$  flat-sky patches obtained by gnomonic projection at angular sampling  $\theta_{\text{patch}} = 6.7'/\text{pixel}$ , giving a linear extent

$$L = N_{\text{patch}} \theta_{\text{patch}} \simeq 7.15^\circ \quad \Rightarrow \quad L^2 \simeq 51 \text{ deg}^2, \quad (37)$$

i.e.  $f_{\text{sky}}^{\text{patch}} \simeq 0.12\%$  per patch. Curvature corrections are of order  $L^2 \sim 1.5 \times 10^{-2}$  and the flat-sky approximation is accurate. Patch projection is implemented by `healpy.projector.GnomonicProj`: the temperature channel is resampled by scalar interpolation, while  $(Q, U)$  are first resampled and then spin-2 rotated to the local patch frame using Eq. (34)'s spin-2 matrix, with rotation angle  $\psi(\hat{n})$  computed numerically by finite-differencing the projection along the local north direction (function `_project_patch_tqu`, flag `rotate_pol=True`; Sec. 3.6). Non-finite values introduced at projection edges are explicitly replaced by zero.

- Patch-centre selection and full-patch validity: candidate centres are drawn from the set  $\{i : M_i \geq 0.9\}$  at  $N_{\text{side}} = 512$ . For each candidate, the mask is itself first projected onto the same  $64 \times 64$  gnomonic grid (function `_project_patch_scalar`), and the candidate is rejected unless every pixel of the projected footprint satisfies  $M \geq 0.9$  (function `_extract_random_patches`). This produces an ensemble of effectively unmasked patches at the cost of a non-trivial reduction of usable sky, with an empirically observed acceptance rate of order 50–60% at  $N_{\text{side}} = 512$  for the 143 GHz mask. A target of  $N_{\text{patches/map}}$  patches is collected per full-sky realisation via rejection sampling, with a hard cap of  $40 N_{\text{patches/map}}$  attempts before raising an error.
- Splits and standardisation: the train/validation split is performed at the level of full-sky realisations rather than at the patch level (a 90/10 split of the  $N_{\text{maps}}$  realisations, followed by concatenation of the corresponding per-realisation patch banks), so that no large-scale Fourier modes are shared between training and validation; per-channel statistics  $\mu_c, \sigma_c$

( $c \in \{T, Q, U\}$ ) are computed once on the training split only and applied verbatim to the validation split (Sec. 3.7, 4.4). To guard against degenerate channels,  $\sigma_c$  is floored at  $10^{-6}$  before inversion.

- Bandpower binning: when validating a learned prior with the flat-sky estimator (34), we bin into  $\sim 30$  linear or log-spaced annuli of  $|\ell|$  between  $\ell_{\min} \simeq 2\pi/L \approx 50$  and  $\ell_{\max} \simeq \pi/\theta_{\text{patch}} \approx 1500$ .

| Parameter                                    | Value                                             |
|----------------------------------------------|---------------------------------------------------|
| HEALPix resolution $N_{\text{side}}$         | 512                                               |
| Harmonic cutoff $\ell_{\max}$                | 1535                                              |
| Common effective beam $\theta_{\text{FWHM}}$ | 10' Gaussian                                      |
| Frequency                                    | 143 GHz                                           |
| Sky mask                                     | Planck 143 GHz confidence, $M \geq 0.9$           |
| Patch grid                                   | $64 \times 64$                                    |
| Patch sampling $\theta_{\text{patch}}$       | 6.7'/pixel                                        |
| Patch extent $L$                             | $\simeq 7.15^\circ$ ( $\simeq 51 \text{ deg}^2$ ) |
| Train/val split                              | 90/10                                             |
| Bandpower bins                               | $\sim 30$ , $\ell \in [50, 1500]$                 |
| Units                                        | $\mu\text{K}_{\text{CMB}}$                        |

Table 1: Summary of common processing parameters used uniformly for the observed sky  $y$ , the simulated CMB prior  $p(f)$  (Sec. 4.2), and the simulated foreground prior  $p(x)$  (Sec. 4.3).

The remainder of this section documents the product-specific processing on top of these common conventions: the Planck half-ring preprocessing (Sec. 4.1), the CAMB-based CMB simulation (Sec. 4.2), and the PySM-based foreground simulation (Sec. 4.3).

#### 4.1 Observed CMB data

We use the publicly released Planck PR4/NPIPE [5] half-ring maps at 143 GHz,  $h_1$  and  $h_2$ , and the corresponding 143 GHz confidence mask, all distributed as HEALPix RING-ordered FITS files at  $N_{\text{side}}^{\text{in}} = 2048$ . Each half-ring file contains the three Stokes parameters ( $I, Q, U$ ) in  $\text{K}_{\text{CMB}}$ .

The two half-ring maps are processed independently by the routine `preprocess_pr4_hr_to_nside512`, which implements the harmonic-space “smooth-and-downgrade” procedure of Sec. 3.2:

1. read  $(I, Q, U)$ , replace **HEALPix UNSEEN** sentinels and non-finite values by zero, and rescale by  $10^6$  to convert  $\text{K}_{\text{CMB}} \rightarrow \mu\text{K}_{\text{CMB}}$  (units inferred from the FITS TUNIT/BUNIT header or, as a fallback, from the map RMS);
2. compute polarised harmonic coefficients ( $a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B$ ) with `healpy.map2alm` (`pol=True`, `iter=3`) up to  $\ell_{\max} = 3N_{\text{side}} - 1 = 1535$ ;
3. multiply each coefficient by the residual Gaussian transfer function

$$B_\ell^{\text{add}} = \exp\left[-\frac{1}{2}\ell(\ell+1)\sigma_{\text{add}}^2\right], \quad \sigma_{\text{add}} = \frac{1}{\sqrt{8\ln 2}}\sqrt{\theta_{\text{out}}^2 - \theta_{\text{in}}^2}, \quad (38)$$

with  $\theta_{\text{in}} = 7.27'$  and  $\theta_{\text{out}} = 10'$  as in Sec. 3.3;

4. synthesise  $(T, Q, U)$  at  $N_{\text{side}} = 512$  via `healpy.alm2map` (`pol=True`).

Pixel-window deconvolution is omitted at this stage, an approximation justified in Sec. 4: the input pixel-window correction at  $N_{\text{side}}^{\text{in}} = 2048$  is  $\lesssim 0.3\%$  throughout the represented multipole range.

From the two preprocessed half-ring maps we form the half-mission-style sum and difference of Sec. 3.8,

$$y = \frac{1}{2}(h_1 + h_2), \quad d = \frac{1}{2}(h_1 - h_2), \quad (39)$$

and use  $y$  as the observed sky entering the likelihood of Sec. 1 and  $d$  as a sky-signal-free realisation of the instrumental noise. On the clean-sky footprint defined in Sec. 4 we verify the expected coherence properties:  $h_1$  and  $h_2$  are positively correlated pixel-by-pixel in all three Stokes channels; the half-ring difference satisfies  $\text{RMS}(d) \ll \text{RMS}(y)$ , with the noise-to-signal RMS ratio of order 10% in  $T$  and order 1 in  $Q, U$  at the  $10'$  resolution; and the binned cross-spectrum  $\hat{C}_\ell^{h_1 \times h_2}$  tracks the fiducial CMB spectrum (Sec. 4.2) across  $30 \leq \ell \leq \ell_{\text{max}}$  in all four diagonal channels  $TT/EE/BB/TE$ . These checks fix the data-domain conventions used identically for the two simulated priors below.

## 4.2 $p(f)$ : simulated CMB priors

The CMB prior  $p(f)$  is constructed at the level of *maps* that are physically lensed by an independently sampled realisation of the lensing potential  $\phi$ , rather than by drawing Gaussian fields with the lensed angular power spectrum directly. This sub-section spells out the simulation pipeline, motivates the choice over the simpler Gaussian-from-lensed- $C_\ell$  alternative (which is retained as a legacy/ablation path), and discusses the path towards a fully self-consistent treatment of lensing as an explicit forward-model operator.

**Cosmology and CAMB call.** We adopt a fiducial flat  $\Lambda$ CDM cosmology with  $H_0 = 67.5 \text{ km s}^{-1} \text{ Mpc}^{-1}$ ,  $\Omega_b h^2 = 0.022$ ,  $\Omega_c h^2 = 0.122$ ,  $\sum m_\nu = 0.06 \text{ eV}$ ,  $\tau = 0.06$ ,  $A_s = 2.1 \times 10^{-9}$ ,  $n_s = 0.965$ ,  $r = 0$ . A single call to the Boltzmann solver CAMB [4] with `lens_potential_accuracy=1` provides, in  $\mu\text{K}^2$  and on the integer grid  $L = 0, \dots, \ell_{\text{max}}$ :

- the unlensed scalar CMB spectra  $\{C_\ell^{TT, \text{u}}, C_\ell^{EE, \text{u}}, C_\ell^{BB, \text{u}}, C_\ell^{TE, \text{u}}\}$  via `get_cmb_power_spectra(..., raw_cl=True)['unlensed_scalar']`; with  $r = 0$  the unlensed  $C_\ell^{BB, \text{u}}$  is identically zero;
- the lensing-potential auto-spectrum  $C_L^{\phi\phi}$  via `get_lens_potential_cls(..., raw_cl=True)[: , 0]`;
- for diagnostic purposes only, the lensed total spectra  $\{C_\ell^{TT}, C_\ell^{EE}, C_\ell^{BB}, C_\ell^{TE}\}$ , which provide the analytic 2-point target that the lensed sims must match in expectation.

The unlensed spectra plus  $C_L^{\phi\phi}$  are the only inputs to the lensing pipeline below; the lensed CAMB spectra are retained as a validation target only.

**Lensed-map sampler.** We use the curved-sky CMB-lensing remapping engine `lenspyx` [21], which wraps the high-performance non-uniform spherical-harmonic transforms of DUCC and computes the deflected sky to a user-controlled accuracy with cost essentially equal to two standard pol-aware SHTs. Each independent map realisation in the prior bank is generated by the routine `simulate_lensed_cmb_tqu_lenspyx`, which proceeds in five harmonic-space steps:

1. *Bandlimits.* Set the lensing-remap bandlimit to  $\ell_{\text{max}} = 3N_{\text{side}} - 1 = 1535$  and zero the unphysical  $\ell = 0, 1$  entries of  $C_\ell^{TT, \text{u}}, C_\ell^{EE, \text{u}}, C_\ell^{BB, \text{u}}, C_\ell^{TE, \text{u}}, C_L^{\phi\phi}$  before any sampling.
2. *Unlensed primary draw.* Draw a single correlated polarised realisation of the unlensed CMB,

$$(a_{\ell m}^{T, \text{u}}, a_{\ell m}^{E, \text{u}}, a_{\ell m}^{B, \text{u}}) \sim \mathcal{N} \left( 0, \begin{pmatrix} C_\ell^{TT, \text{u}} & C_\ell^{TE, \text{u}} & 0 \\ C_\ell^{TE, \text{u}} & C_\ell^{EE, \text{u}} & 0 \\ 0 & 0 & C_\ell^{BB, \text{u}} \end{pmatrix} \right), \quad (40)$$

with `healpy.synalm(new=True)`, exactly as in the legacy Gaussian sampler but with the *unlensed* CAMB spectra.

3. *Lensing-potential draw.* Independently draw a Gaussian realisation of the lensing potential,

$$\phi_{LM} \sim \mathcal{N}(0, C_L^{\phi\phi}), \quad (41)$$

by a separate `healpy.synalm` call. The CMB $\times\phi$  cross-spectra  $C_L^{\phi T}$  and  $C_L^{\phi E}$  are not imposed here, an approximation discussed below.

4. *Deflection field.* The gravitational deflection vector is the gradient of  $\phi$ ,  $\alpha(\hat{n}) = \nabla\phi(\hat{n})$ , with curl-free gradient-mode alm  $d_{LM}^{\text{grad}} = \sqrt{L(L+1)}\phi_{LM}$  (curl-mode  $\Omega_{LM} = 0$  for scalar lensing). The factor  $\sqrt{L(L+1)}$  is precisely the `lenspyx` convention for the input deflection alm and is applied via `healpy.almxfl`.
5. *Remap onto the deflected sphere.* A single call

$$\text{lenspyx.alm2lenmap}([a^{T,u}, a^{E,u}, a^{B,u}], [d^{\text{grad}}])$$

with `geometry=('healpix', {nside: 512})`, `pol=True` and `epsilon=10-6` produces a *truly lensed* ( $T_{\text{lens}}, Q_{\text{lens}}, U_{\text{lens}}$ ) realisation: the remap is implemented by `ducc0` as a pair of non-uniform SHTs that evaluate the unlensed CMB at the deflected positions and rotate ( $Q, U$ ) by the appropriate spin-2 phase along the deflection direction.

The output of step (5) is a HEALPix map at  $N_{\text{side}} = 512$  that carries the full lensing imprint of the sampled  $\phi$ , not just its diagonal power-spectrum signature. By construction it inherits, in expectation, the lensed CAMB power spectra of step (1), but at the level of an individual realisation it also contains all the lensing-induced non-Gaussian structure: the connected 4-point function (trispectrum) that underpins quadratic CMB-lensing reconstruction, the non-Gaussian texture of lensing  $B$ -modes generated by  $E \rightarrow B$  mode coupling, and the lensing modulation of the small-scale acoustic structure (Sec. 3).

**Post-lensing instrumental response.** The remap is performed at the native unbeamed, unwindowed resolution. The common observational response is then applied to the lensed map in harmonic space, matching the convention used for  $y$  in Sec. 4.1:

1. compute  $(a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B)$  of  $(T_{\text{lens}}, Q_{\text{lens}}, U_{\text{lens}})$  via `healpy.map2alm (pol=True, iter=3)`;
2. multiply by the joint beam-and-pixel-window transfer functions

$$a_{\ell m}^T \rightarrow B_\ell P_\ell^T a_{\ell m}^T, \quad a_{\ell m}^{E,B} \rightarrow B_\ell P_\ell^P a_{\ell m}^{E,B}, \quad (42)$$

with  $B_\ell$  from Eq. (24) at  $\theta_{\text{FWHM}} = 10'$  and  $P_\ell^{T,P}$  from `healpy.pixwin(., pol=True)`;

3. synthesise the final  $(T, Q, U)$  at  $N_{\text{side}} = 512$  via `healpy.alm2map (pol=True)`.

This ordering—lens *first*, beam and pixel-window *after*—is the physically correct one: the beam and the HEALPix pixel window act on the observed (and therefore lensed) sky, not on the underlying primary CMB. Each realisation is generated from an independent random seed that drives both the unlensed  $(a^{T,u}, a^{E,u}, a^{B,u})$  draw and the independent  $\phi$  draw, so  $N_f$  runs yield a statistically independent ensemble at fixed cosmology.

**Why not just Gaussian draws from the lensed  $C_\ell$ ?** The legacy sampler `simulate_cmb_tqu_from_cls`, which draws Gaussian alm from the *lensed* CAMB spectra and applies beam/pixel-window in one shot, is retained in the codebase for back-compatibility and as a controlled ablation. It is however strictly mis-specified once any non-2-point statistic is exposed to inference:

- It reproduces  $\langle C_\ell^{TT/EE/BB/TE} \rangle$  by construction but has identically Gaussian one-point and higher-order  $n$ -point statistics, with vanishing lensing trispectrum and vanishing  $T, E, \phi$ -bispectrum at fixed  $\phi$ .

- The lensing-induced  $B$ -modes generated by  $E \rightarrow B$  leakage have, in real lensed skies, a characteristic non-Gaussian morphology that traces the underlying  $\phi$  field; a Gaussian draw from the same  $C_\ell^{BB}$  produces  $B$ -modes with no such structure and is observationally indistinguishable from primordial  $r$ -driven  $B$  at the 2-point level only.
- Cosmology-marginalisation extensions (varying  $\Omega_m$ ,  $\Omega_\Lambda$ , etc.) shift both  $C_\ell^{\text{unl}}$  and  $C_L^{\phi\phi}$  *consistently*, so the lenspyx pipeline correctly propagates parameter variations to the lensed CMB; the Gaussian-lensed- $C_\ell$  pipeline only propagates the marginal  $C_\ell^{\text{lensed}}$ , losing the correlated parameter dependence of the lensing kernel.

The remapping cost is dominated by two pol-aware SHTs at  $N_{\text{side}} = 512$  (a fraction of a second per realisation on a single core through DUCC), so the upgrade is essentially free relative to map I/O and patch extraction. Empirically (notebook validation cell), individual lenspyx realisations recover the lensed CAMB  $\{C_\ell^{TT}, C_\ell^{EE}\}$  to within cosmic variance on the full sky; the lensing-induced  $C_\ell^{BB}$  converges from below as the lensing bandwidth approaches the support of  $C_L^{\phi\phi} \otimes C_\ell^{EE}$ , an artefact of the finite- $\ell_{\text{max}}$  remap that vanishes well before the noise level of Planck 143 GHz.

**Approximations and forward look: explicit lensing operator (Option 2).** The current pipeline carries one explicit approximation: the unlensed CMB ( $a^{T,u}, a^{E,u}, a^{B,u}$ ) and the lensing potential  $\phi_{LM}$  are drawn *independently*, so the small ( $\mathcal{O}(10^{-3})$ ) CMB $\times\phi$  cross-spectra  $C_L^{\phi T}$  (from late-time ISW) and  $C_L^{\phi E}$  are not imposed at the level of individual realisations. This is the standard convention used by Planck FFP10 [5] and CMB-S4 simulation libraries, and is negligible compared with cosmic variance at the multipole range used here ( $\ell \lesssim 1500$ ). A subdominant second approximation is that  $\phi$  is treated as Gaussian; the leading-order  $\phi$  non-Gaussianity from non-linear large-scale structure is at the few-percent level on  $C_L^{\phi\phi}$  [22] and well below the simulation accuracy required for the present problem.

A more principled treatment, planned as a follow-up extension when polarisation  $B$ -modes (and hence the tensor-to-scalar ratio  $r$ ) become the science target, is to absorb the lensing remap into the forward model rather than into the prior. Concretely, Eq. (1) is rewritten as

$$y = A [\mathcal{L}(\phi) f_u + x] + n, \quad (43)$$

where  $f_u$  is the *unlensed* CMB and  $\mathcal{L}(\phi)$  is the gravitational-lensing operator built from the lensing potential  $\phi$  (the same operator that `lenspyx.alm2lenmap` implements). The advantages of this “Option 2” are that

1. the prior  $p(f_u)$  becomes a Gaussian on the unlensed CMB, which is the physically correct prior and whose density is closed-form—no need to train a neural prior to learn lensing non-Gaussianity, since it is now in the model;
2. the lensing potential  $\phi$  becomes a latent random field with a Gaussian prior  $p(\phi) = \mathcal{N}(0, C_L^{\phi\phi})$  that can be jointly inferred from the data, yielding a calibrated posterior  $p(f_u, x, \phi | y)$  rather than a marginal  $p(f, x | y)$ ;
3. cosmological-parameter marginalisation is straightforward because all dependence on  $\Omega_m$ ,  $\Omega_\Lambda$ ,  $H_0$  is now carried by the closed-form Gaussian priors on  $f_u$  and  $\phi$  rather than by an empirically learned non-Gaussian prior.

Three sub-variants of Option 2 are natural and would re-use most of the present infrastructure:

- *Fixed- $\phi$ .* Replace  $\phi$  in Eq. (43) with the public Planck PR4 lensing reconstruction `COM_Lensing_4096_R4.00` [23]. The likelihood gradients used during posterior sampling become  $\nabla_{f_u} \log p(y | f_u, x)$  computed through the linear adjoint of  $\mathcal{L}(\hat{\phi})$ , which lenspyx also exposes. Cheapest variant but ignores reconstruction noise on  $\hat{\phi}$ .

- *Joint  $\phi$  sampling.* Augment the latent state of the diffusion sampler with  $\phi$  and add likelihood gradients  $\nabla_{\phi} \log p(y \mid f_u, x, \phi)$  alongside the existing  $\nabla_{f_u, x} \log p(y \mid f_u, x, \phi)$ . This realises the MUSE / Anderes–Millea–Seljak scheme [24] in the diffusion-sampler framework and yields a fully calibrated joint posterior on  $(f_u, x, \phi)$ .
- *Quadratic-marginalised  $\phi$ .* Marginalise  $\phi$  to leading order analytically (Hu–Okamoto quadratic estimator). Useful as a fast consistency check; the residual non-Gaussianity in  $f_u$  after marginalisation is at the level of higher-order  $\phi$  moments and is negligible at Planck noise levels.

The Option-1 sampler implemented in this work is the strict subset of the fixed- $\phi$  variant in which one further drops the dependence on  $\hat{\phi}$  at sampling time and absorbs the resulting marginal lensed CMB into a learned prior. It is the right starting point for the present temperature-dominated analysis, and provides a precise quantitative baseline against which any future Option-2 extension will be validated.

**Output.** A bank of  $N_f$  independent full-sky lenspyx-lensed realisations is generated by the routine `build_prior_patch_datasets` (with `lensing_method='lenspyx_remap'`, the default) and reduced to flat  $(T, Q, U)$  patches by the common patch-extraction and standardisation pipeline of Sec. 4. The resulting training-ready array has shape  $(N_{\text{patches}}, 3, 64, 64)$  with  $N_{\text{patches}} = N_f \times N_{\text{patches}/\text{map}}$ . Selecting `lensing_method='gaussian_lensed_cl'` switches back to the legacy Gaussian-from-lensed- $C_\ell$  sampler with the same output schema, useful as an ablation control.

### 4.3 $p(x)$ : simulated foreground priors

The Galactic-foreground prior is built from PySM3 [6, 7], instantiated at  $N_{\text{side}} = 512$  with the small-scale component model `d10+s5`: `d10` is a modified-blackbody thermal-dust template whose dust temperature  $T_d(\hat{n})$  and spectral index  $\beta_d(\hat{n})$  are spatially varying maps derived from Planck data, and whose small-scale fluctuations below the resolution of the input templates ( $\ell \gtrsim 300$ ) are populated stochastically by a non-Gaussian small-scale model fitted to high-latitude Planck residuals; `s5` is the analogous spatially-varying-spectral-index synchrotron model whose small-scale modes are drawn stochastically from a fitted power-law amplitude. The choice of small-scale presets, rather than the deterministic `d1+s1` pair, is critical: it makes each foreground realisation a genuinely independent draw of the small-scale Galactic field, rather than a rotated copy of a single deterministic template, and is what allows the learned prior  $p(x)$  to capture the patch-level variability of the Galactic emission. We exclude PySM’s `c1` (lensed-CMB) component from the foreground sky used in  $p(x)$ , since the lensed CMB is already drawn independently as  $p(f)$  and including it twice would bias the joint prior; an exploratory `d1+s1+c1` sky is instantiated at the top of the data section purely as a visual sanity check (Mollweide views of  $I$  and  $P = \sqrt{Q^2 + U^2}$  at 143 GHz) and is not used downstream.

**Independent stochastic realisations.** For each of  $N_x$  realisations we re-seed the random number generator that drives PySM’s small-scale models with a fresh seed derived deterministically from a master foreground seed, and then build a new `Sky` object. Because PySM samples its small-scale modes at `Sky`-construction time, this guarantees that successive realisations have *statistically independent* small-scale Galactic structure while sharing the same large-scale template above the small-scale-model crossover multipole. In particular, both the morphology of the small-scale dust filaments and the small-scale polarisation amplitude vary realisation-to-realisation, in contrast with the deterministic `d1+s1` presets where these would be repeated identically. Emission is evaluated at the common observing frequency 143 GHz and converted from PySM’s native Rayleigh–Jeans antenna units to thermodynamic  $\mu K_{\text{CMB}}$  through the Planck blackbody equivalency, ensuring unit consistency with  $y$  and  $f$ .

**Random orientation augmentation.** Each stochastic realisation is then optionally rotated by a uniformly sampled element  $R \in \text{SO}(3)$ , drawn via Euler angles

$$\alpha \sim \mathcal{U}(0, 2\pi), \quad \cos \beta \sim \mathcal{U}(-1, 1), \quad \gamma \sim \mathcal{U}(0, 2\pi), \quad (44)$$

acting on the full  $(T, Q, U)$  stack with the spin-correct rotation of polarisation under change of basis (so that  $E/B$  are preserved). Because the Galactic plane is a strong morphological prior on the unrotated PySM sky, the rotation augmentation prevents the patch ensemble from being biased towards the small subset of orientations sampled by a fixed Galactic frame, and (combined with the by-map train/validation split below) avoids the trivial overfitting in which the model memorises specific Galactic features.

**Common transfer-function application.** The rotated  $(T, Q, U)$  realisation is then transported into harmonic space, multiplied by the common transfer functions, and synthesised back to a HEALPix map. Specifically, polarised harmonic coefficients  $(a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B)$  up to the band-limit  $\ell_{\text{max}} = 3N_{\text{side}} - 1$  are filtered through

$$a_{\ell m}^T \rightarrow B_\ell P_\ell^T a_{\ell m}^T, \quad a_{\ell m}^{E,B} \rightarrow B_\ell P_\ell^P a_{\ell m}^{E,B}, \quad (45)$$

with the  $10'$  FWHM Gaussian beam  $B_\ell$  and the polarisation-aware HEALPix pixel windows  $P_\ell^T$ ,  $P_\ell^P$  defined in Sec. 3.3–3.4. This yields an effective transfer function for  $p(x)$  that is *identical* to the one applied to  $p(f)$  in Sec. 4.2, which is a prerequisite for any well-posed joint inference of  $f$  and  $x$  from the same observed  $y$ : a mismatch in the effective beams or pixel windows of the two priors would propagate as a bias on the relative amplitudes of the two components in the posterior.

The full pipeline for a single foreground realisation is therefore

$$\underbrace{\text{PySM d10+s5@143 GHz}}_{\text{re-seeded Sky, independent draw}} \xrightarrow{\text{SO(3) rotation}} \xrightarrow{\text{beam + pixel window}} \xrightarrow{\text{patches}} \xrightarrow{\text{by-map split + standardise}} \text{saved .npz}. \quad (46)$$

Patch extraction, splitting and per-channel standardisation proceed identically to  $p(f)$  via the common pipeline of Sec. 4, ensuring that the two priors are ingested by the U-Net in the same numerical and angular domain. This is a prerequisite for the joint posterior sampler of Sec. 1, which assumes a single shared representation for  $f$  and  $x$ .

**Limitations.** We note two limitations of this construction inherited from the underlying simulation choices. First, although d10+s5 populates the small scales stochastically, the large-scale structure ( $\ell \lesssim 300$ ) is still anchored to a deterministic Planck-derived template; the rotation augmentation enriches the orientation distribution but not the large-scale stochasticity of the Galactic field, so the learned prior  $p(x)$  is best interpreted as the distribution of 143 GHz foreground patches consistent with the Planck-derived large-scale templates and a fitted small-scale model, rather than a fully marginalised model over Galactic-emission parameters. Second, additional foreground species (AME, free-free, point sources) are deferred to future work. These restrictions are consistent with the scope of the present prior-learning stage and do not affect the validity of the pipeline that follows.

#### 4.4 Conversion to flat patches and saved prior datasets

We collect here, in one place, the full sequence of transformations applied to each prior between simulation and disk, together with the on-disk format of the resulting datasets. This is the data that is loaded by the diffusion-model training and sampling cells of Sec. 5, and the description that follows fixes everything the reader needs to know to reproduce them.

**Per-realisation pipeline.** For a single full-sky realisation of  $f$  or  $x$ , the transformations are applied in the following fixed order:

1. *Generate* a  $(T, Q, U)$  HEALPix map at  $N_{\text{side}} = 512$  in  $\mu\text{K}_{\text{CMB}}$ , including all relevant transfer functions. For  $p(f)$ , the default lensing strategy is the curved-sky lenspyx remap of Sec. 4.2: an independent unlensed CMB realisation  $(a^T, a^E, a^B)$  is drawn from the unlensed CAMB spectra together with an independent realisation  $\phi_{LM}$  of the lensing potential, the unlensed sky is gravitationally lensed by  $\phi$  on the curved sphere, and the common Gaussian beam  $B_\ell$  and HEALPix pixel windows  $P_\ell^{T,P}$  are applied in harmonic space to the lensed map. The legacy Gaussian-from-lensed- $C_\ell$  path shares the same beam-and-pixel-window step and remains available as an ablation control. For  $p(x)$ , each realisation re-seeds PySM’s small-scale random number generator and instantiates a fresh **Sky**, so that the small-scale Galactic structure is statistically independent across realisations (Sec. 4.3); the resulting full-sky map is rotated by a uniform  $\text{SO}(3)$  element and then propagated through the same harmonic-space beam-and-pixel-window step used for  $p(f)$ , so that the effective transfer functions of the two priors are identical by construction.
2. *Patch extraction* via `_extract_random_patches`: a target of  $N_{\text{patches}/\text{map}}$  flat-sky patches is collected by rejection sampling, with candidate centres restricted to mask-clean HEALPix pixels and the full  $64 \times 64$  projected footprint required to satisfy  $M \geq 0.9$  (Sec. 4). Each accepted patch is the output of `_project_patch_tqu` with `rotate_pol=True`, which scalar-resamples  $T$ , scalar-resamples  $(Q, U)$ , and then spin-2 rotates  $(Q, U)$  to the local patch frame using a numerically-evaluated  $\psi(\hat{n})$  (Sec. 3.6); non-finite values introduced at patch edges are replaced by zero.
3. *Concatenation* of the per-realisation patch banks into a single array of shape  $(N_{\text{patches}}, 3, 64, 64)$  with  $N_{\text{patches}} = N_{\text{maps}} \times N_{\text{patches}/\text{map}}$ , dtype `float32`, channel order  $(T, Q, U)$ .

**Train/validation split by map index.** The split is performed at the level of *full-sky realisations*, not at the level of individual patches. Concretely, the  $N_{\text{maps}}$  underlying full-sky realisations are first shuffled with a deterministic seed and partitioned 90/10 into a training set of  $N_{\text{maps}}^{\text{train}}$  realisations and a validation set of  $N_{\text{maps}}^{\text{val}}$  realisations; the corresponding per-realisation patch banks are then concatenated separately into `train_raw` and `val_raw`. This is essential. A patch-level split would routinely place patches drawn from the *same* underlying realisation into both the training and validation sets, in which case the validation set would inherit the large-scale Fourier modes already seen at training time (a non-trivial fraction of the patch power lies at  $\ell \lesssim \pi/L \approx 50$ , where  $L$  is the patch extent). The validation diagnostics would then systematically underestimate the generalisation error, and any spectral checkpoint selection driven by validation metrics would be biased. Splitting by realisation enforces statistical independence between the two splits at all scales.

**Per-channel standardisation.** The two raw arrays are then passed through a deterministic per-channel affine standardisation:

1. compute per-channel statistics

$$\mu_c = \text{mean}_{(\text{patch}, x, y)}(\text{train\_raw}_c), \quad \sigma_c = \text{std}_{(\text{patch}, x, y)}(\text{train\_raw}_c), \quad c \in \{T, Q, U\}, \quad (47)$$

on the training split only, with  $\sigma_c$  floored at  $10^{-6}$  before inversion to guard against degenerate channels;

2. apply the affine map  $\tilde{X}_c = (X_c - \mu_c)/\sigma_c$  to both splits to produce `train_norm` and `val_norm`;
3. shuffle the patches *within* each split, with an independent seed, to break the per-realisation block ordering inherited from the by-map concatenation.

**On-disk format.** The two priors are written to disk in compressed NumPy archives,

$$\text{cmb\_fg\_prior\_f\_cmb.npz}, \quad \text{cmb\_fg\_prior\_x\_fg.npz}, \quad (48)$$

each containing six float32 arrays:

| Key                     | Description                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------|
| <code>train_raw</code>  | Training patches in physical $\mu\text{K}_{\text{CMB}}$ , shape $(N_{\text{train}}, 3, 64, 64)$ |
| <code>val_raw</code>    | Validation patches in physical $\mu\text{K}_{\text{CMB}}$ , shape $(N_{\text{val}}, 3, 64, 64)$ |
| <code>train_norm</code> | Standardised training patches, shape $(N_{\text{train}}, 3, 64, 64)$                            |
| <code>val_norm</code>   | Standardised validation patches, shape $(N_{\text{val}}, 3, 64, 64)$                            |
| <code>mean</code>       | Per-channel training mean $\mu_c$ , shape $(1, 3, 1, 1)$                                        |
| <code>std</code>        | Per-channel training std $\sigma_c$ , shape $(1, 3, 1, 1)$                                      |

A companion human-readable `cmb_fg_meta.json` captures every parameter of the build (resolution,  $\ell_{\text{max}}$ , beam, pixel-window flag, patch geometry, mask threshold, full-patch validity flag,  $N_{\text{maps}}$ , foreground frequency and presets, rotation flag, train fraction, and all RNG seeds), so that the dataset is fully reproducible from the notebook state.

Storing both `*_raw` (physical units) and `*_norm` (standardised) variants serves two complementary purposes: training and U-Net inference are performed entirely in the normalised domain (where  $T, Q, U$  have unit per-channel variance and zero mean by construction), while spectral diagnostics, posterior sampling, and any comparison against theoretical  $C_\ell$  tables are most naturally expressed in physical  $\mu\text{K}_{\text{CMB}}^2$  units and require the de-standardisation  $X_c = \sigma_c \tilde{X}_c + \mu_c$ , with  $\mu_c, \sigma_c$  retrieved from the saved `mean, std` arrays.

This concludes the data description. Everything entering the diffusion-model training and posterior-sampling stages of Sec. 5 is fully specified by these two `.npz` files together with the original  $y, d$  half-ring maps of Sec. 4.1, the common processing parameters of Table 1, and the metadata file written alongside the priors.

## 5 Deterministic flow matching for patch-based CMB priors

The remainder of this note is devoted to the generative-model component of the pipeline. We learn the priors  $p(f)$  and  $p(x)$  on the standardised patch space

$$\mathbf{x} \in \mathbb{R}^D, \quad D = C H W = 3 \times 64 \times 64 = 12,288,$$

by training, separately for each prior, a continuous-time generative model that transports samples from a tractable base distribution  $p_{\text{init}} = \mathcal{N}(0, I_D)$  to the empirical patch distribution  $p_{\text{data}}$  along an ODE driven by a learned vector field. We use the *deterministic* flow-matching formulation of [10, 32, 33], which makes sampling a noiseless map from base noise to data and is therefore the natural choice for an analytic prior to be plugged into a downstream posterior sampler with explicit likelihood gradients (Sec. 5.6). For completeness we also explain how the same machinery degenerates, with  $\sigma > 0$ , into the score-augmented SDE family that includes denoising diffusion as a special case, but emphasise that the production model in this work is run as a pure deterministic flow.

### 5.1 Mathematical formalism: probability paths and flow matching

#### 5.1.1 Probability paths and flow models

**Definition 5.1** (Probability path). A *probability path* on  $\mathbb{R}^D$  is a continuous family of probability densities  $\{p_t\}_{t \in [0,1]}$  that interpolates between a tractable base  $p_0 = p_{\text{init}}$  and the data distribution  $p_1 = p_{\text{data}}$ . We write  $p_t(\mathbf{x})$  for the density of the marginal at time  $t$ .

**Definition 5.2** (Marginal velocity field). A *marginal velocity field* for the probability path  $\{p_t\}$  is a measurable map  $u : [0, 1] \times \mathbb{R}^D \rightarrow \mathbb{R}^D$  such that the deterministic ODE

$$\mathbf{x}_0 \sim p_{\text{init}}, \quad \frac{d\mathbf{x}_t}{dt} = u_t(\mathbf{x}_t) \quad (49)$$

generates  $\mathbf{x}_t \sim p_t$  for every  $t \in [0, 1]$ .

The fundamental link between the path  $\{p_t\}$  and the field  $u_t$  is the continuity equation, which expresses local conservation of probability along the flow:

$$\partial_t p_t(\mathbf{x}) + \nabla_{\mathbf{x}} \cdot [u_t(\mathbf{x}) p_t(\mathbf{x})] = 0, \quad t \in [0, 1], \quad (50)$$

interpreted as a partial differential equation in the distributional sense [10]. Conversely, given any path  $\{p_t\}$  for which Eq. (50) admits a smooth solution  $u_t$  with  $u_t(\mathbf{x})$  bounded, the ODE in Eq. (49) defines the unique flow  $\psi_t : \mathbb{R}^D \rightarrow \mathbb{R}^D$  such that  $\psi_t(\mathbf{x}_0) \sim p_t$  for  $\mathbf{x}_0 \sim p_{\text{init}}$ . Sampling from  $p_{\text{data}}$  then reduces to numerical integration of Eq. (49) up to  $t = 1$ .

The strategic choice of [10] is to design the path  $\{p_t\}$  first, then *learn*  $u_t$  by regression. The difficulty is that even when  $\{p_t\}$  is given by a closed-form recipe, the marginal  $p_t$  is in general only available through a high-dimensional integral over the data distribution and the generating noise. Direct simulation-based regression of  $u_t$  is therefore intractable. The *conditional flow-matching* construction circumvents this by introducing a conditioning variable.

### 5.1.2 Conditional probability path and the marginalisation theorem

**Definition 5.3** (Conditional probability path). Let  $q$  be a fixed distribution on a measurable conditioning space  $\mathcal{Z}$ . A *conditional probability path* indexed by  $q$  is a family  $p_t(\mathbf{x} \mid \mathbf{z})$  of densities on  $\mathbb{R}^D$ , parametrised by  $\mathbf{z} \sim q$ , together with a *conditional velocity field*  $u_t(\mathbf{x} \mid \mathbf{z})$  that satisfies the conditional continuity equation

$$\partial_t p_t(\mathbf{x} \mid \mathbf{z}) + \nabla_{\mathbf{x}} \cdot [u_t(\mathbf{x} \mid \mathbf{z}) p_t(\mathbf{x} \mid \mathbf{z})] = 0 \quad \forall \mathbf{z} \in \mathcal{Z}. \quad (51)$$

We take the conditioning distribution to be the data distribution itself,  $q = p_{\text{data}}$ , and design  $p_t(\mathbf{x} \mid \mathbf{z})$  so that  $p_0(\mathbf{x} \mid \mathbf{z}) = p_{\text{init}}(\mathbf{x})$  for every  $\mathbf{z}$  and  $p_1(\mathbf{x} \mid \mathbf{z}) = \delta(\mathbf{x} - \mathbf{z})$  in the distributional sense. The marginal path is then recovered by

$$p_t(\mathbf{x}) = \int_{\mathcal{Z}} p_t(\mathbf{x} \mid \mathbf{z}) p_{\text{data}}(\mathbf{z}) d\mathbf{z}, \quad (52)$$

and a direct consequence of integrating Eq. (51) against  $p_{\text{data}}(\mathbf{z}) d\mathbf{z}$  is the following marginalisation theorem.

**Theorem 5.4** (Marginal velocity field; [10]). For any conditional probability path satisfying Definition 5.3, the marginal path Eq. (52) admits the marginal velocity field

$$u_t(\mathbf{x}) = \int_{\mathcal{Z}} u_t(\mathbf{x} \mid \mathbf{z}) \frac{p_t(\mathbf{x} \mid \mathbf{z}) p_{\text{data}}(\mathbf{z})}{p_t(\mathbf{x})} d\mathbf{z} = \mathbb{E}_{\mathbf{z} \sim p_t(\mathbf{z} \mid \mathbf{x})} [u_t(\mathbf{x} \mid \mathbf{z})], \quad (53)$$

with  $p_t(\mathbf{z} \mid \mathbf{x}) \propto p_t(\mathbf{x} \mid \mathbf{z}) p_{\text{data}}(\mathbf{z})$  the time- $t$  posterior of  $\mathbf{z}$  given  $\mathbf{x}$ .

Equation (53) is fundamental: it shows that the unknown marginal velocity field is the posterior expectation of a closed-form quantity. The minimum-mean-square-error (MMSE) estimator of  $u_t(\mathbf{x})$  given  $\mathbf{x}$  is therefore precisely Eq. (53), i.e. the unique minimiser of the regression

$$\min_f \mathbb{E}_{t, \mathbf{z}, \mathbf{x} \mid \mathbf{z}} [\|f(\mathbf{x}, t) - u_t(\mathbf{x} \mid \mathbf{z})\|^2] \quad \text{is} \quad f^*(\mathbf{x}, t) = u_t(\mathbf{x}). \quad (54)$$

### 5.1.3 The conditional flow-matching loss

Theorem 5.4 and Eq. (54) suggest a tractable training objective. We introduce a parametric vector field  $\mathbf{u}_t^\theta : \mathbb{R}^D \times [0, 1] \rightarrow \mathbb{R}^D$  (a neural network with parameters  $\theta$ , see Sec. 5.3) and minimise the *conditional flow-matching loss*

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{z} \sim p_{\text{data}}} \mathbb{E}_{\mathbf{x} \sim p_t(\cdot | \mathbf{z})} \left[ \left\| \mathbf{u}_t^\theta(\mathbf{x}) - u_t(\mathbf{x} | \mathbf{z}) \right\|^2 \right]. \quad (55)$$

**Proposition 5.5** (Equivalence of marginal and conditional flow matching). Let

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, \mathbf{x} \sim p_t} \left[ \left\| \mathbf{u}_t^\theta(\mathbf{x}) - u_t(\mathbf{x}) \right\|^2 \right] \quad (56)$$

be the (intractable) marginal flow-matching loss. Then  $\mathcal{L}_{\text{CFM}}(\theta) - \mathcal{L}_{\text{FM}}(\theta)$  is a constant independent of  $\theta$ , and consequently

$$\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{FM}}(\theta).$$

In particular, the unique global minimiser of  $\mathcal{L}_{\text{CFM}}$  over the population of all measurable maps satisfies  $\mathbf{u}_t^*(\mathbf{x}) = u_t(\mathbf{x})$  almost everywhere along the marginal path.

The proof is a direct consequence of Theorem 5.4: the cross-term  $\mathbb{E}[\mathbf{u}_t^\theta(\mathbf{x}) u_t(\mathbf{x} | \mathbf{z})]$  in  $\mathcal{L}_{\text{CFM}}$  equals  $\mathbb{E}[\mathbf{u}_t^\theta(\mathbf{x}) \cdot u_t(\mathbf{x})]$  once the inner expectation over  $\mathbf{z}$  is evaluated with  $p_t(\mathbf{z} | \mathbf{x})$ , while the difference  $\|u_t(\mathbf{x} | \mathbf{z})\|^2 - \|u_t(\mathbf{x})\|^2$  is  $\theta$ -independent. Proposition 5.5 is what makes the entire flow-matching programme tractable: *minimising a closed-form, low-variance regression target with the same gradient as the intractable marginal regression yields a network whose posterior-mean prediction is the marginal velocity field that drives Eq. (49).*

Conditional flow matching is therefore a generalisation of the classical denoising score-matching argument of [34, 35], replacing the score  $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$  by the velocity  $u_t(\mathbf{x})$ . Both quantities are MMSE projections of a closed-form conditional onto  $\mathbf{x}$ .

### 5.1.4 Score-velocity correspondence and the deterministic limit

For any conditional path one can also define the conditional score  $s_t(\mathbf{x} | \mathbf{z}) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x} | \mathbf{z})$  and, applying Theorem 5.4-style marginalisation, show that the marginal score  $s_t(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$  is similarly given by a posterior expectation. For the Gaussian conditional paths used in this work (Sec. 5.2), the marginal velocity field and the marginal score are linked by a closed-form linear identity

$$u_t(\mathbf{x}) = \left( \beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) s_t(\mathbf{x}) + \frac{\dot{\alpha}_t}{\alpha_t} \mathbf{x}, \quad (57)$$

which means that learning  $u_t$  and learning  $s_t$  are equivalent up to a deterministic post-hoc rescaling [10]. This identity also explains why the entire family of denoising diffusion models [8, 9] can be viewed as a special case of flow matching with a particular schedule choice and a fixed reparametrisation of the regression target.

For *any* smooth  $\sigma_t \geq 0$ , the SDE

$$d\mathbf{x}_t = \left[ u_t(\mathbf{x}_t) + \frac{\sigma_t^2}{2} s_t(\mathbf{x}_t) \right] dt + \sigma_t d\mathbf{W}_t, \quad \mathbf{x}_0 \sim p_{\text{init}}, \quad (58)$$

generates the same marginals  $\{p_t\}$  as the deterministic ODE in Eq. (49) (this is a textbook consequence of the Fokker–Planck equation; see Theorem 17 of [10]). The deterministic flow corresponds to  $\sigma_t \equiv 0$  and the variance-preserving DDPM SDE corresponds to a particular non-zero schedule of  $\sigma_t$ . The two ends of this family are mathematically equivalent statements about the same path  $\{p_t\}$ , but they have very different practical implications. We discuss this trade-off and our deterministic-flow choice next.

### 5.1.5 Why deterministic flow matching for this work

We sample exclusively from the deterministic ODE flow ( $\sigma_t \equiv 0$ ), and use the score-augmented SDE in Eq. (58) only as a theoretical scaffolding to relate our framework to denoising diffusion. Three reasons motivate this choice.

1. *Calibrated, gradient-friendly priors for posterior sampling.* The downstream goal of the project (Sec. 1) is component separation by joint sampling of  $(f, x)$  from  $p(f, x | y) \propto p(y | f, x) p(f) p(x)$ . The natural posterior sampler in this setting is a guided ODE that integrates the prior velocity field plus a likelihood-gradient correction  $\nabla_{(f, x)} \log p(y | f, x)$  at each step [36, 37]. A deterministic flow gives a smooth, fully differentiable map from base noise to sample, so the likelihood-gradient guidance never collides with extra stochastic noise injected by the sampler itself.
2. *Lower-variance training and sampling.* For a Gaussian conditional path with a smooth schedule, the conditional velocity target  $u_t(\mathbf{x} | \mathbf{z})$  has variance bounded by a  $t$ -independent constant, while the equivalent score target diverges as  $\beta_t \rightarrow 0$  (see Sec. 5.2). The CFM loss therefore avoids the per-time-bin reweighting that DDPM-style score matching needs near the data end of the path [8, 26]. Empirically, this manifests as smaller training-loss variance and more stable validation diagnostics (Sec. 5.4).
3. *Half the network evaluations at sampling time.* Because the flow is deterministic, second-order ODE solvers such as Heun (Sec. 5.6) achieve  $\mathcal{O}(h^3)$  local error with two velocity-field evaluations per step. The corresponding stochastic SDE solvers can only reach  $\mathcal{O}(h^2)$  at the same compute budget without specialised splitting schemes.

The price for the deterministic choice is that the sampler is a one-to-one map: distinct base draws  $\mathbf{x}_0 \sim p_{\text{init}}$  are mapped to distinct samples, and there is no in-sampler stochasticity that could mask a residual training imperfection by adding noise. We compensate by using a sufficiently fine integration grid and by validating with the spectral diagnostics of Sec. 5.7.

## 5.2 Probability path: Gaussian interpolant with linear schedule

We instantiate the conditional path of Definition 5.3 with the family of *Gaussian conditional paths* parametrised by two real-valued schedule functions  $\alpha, \beta \in C^1([0, 1])$ :

$$\boxed{p_t(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{x}; \alpha_t \mathbf{z}, \beta_t^2 I_D),} \quad \alpha_t, \beta_t \geq 0, \quad (59)$$

with boundary conditions  $\alpha_0 = 0, \alpha_1 = 1$  and  $\beta_0 = 1, \beta_1 = 0$  that enforce  $p_0(\cdot | \mathbf{z}) = p_{\text{init}} = \mathcal{N}(0, I_D)$  and  $p_1(\cdot | \mathbf{z}) = \delta(\cdot - \mathbf{z})$ . The marginal of Eq. (59) under  $\mathbf{z} \sim p_{\text{data}}$  is the noisy data distribution  $p_t(\mathbf{x}) = \int p_t(\mathbf{x} | \mathbf{z}) p_{\text{data}}(\mathbf{z}) d\mathbf{z}$ , which interpolates between the standard Gaussian at  $t = 0$  and  $p_{\text{data}}$  at  $t = 1$ .

### 5.2.1 Conditional velocity field and noise reparametrisation

A draw from  $p_t(\cdot | \mathbf{z})$  is obtained by the reparametrisation

$$\mathbf{x}_t = \alpha_t \mathbf{z} + \beta_t \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I_D), \quad (60)$$

which directly yields  $\mathbf{x}_t \sim \mathcal{N}(\alpha_t \mathbf{z}, \beta_t^2 I_D)$ . Differentiating Eq. (60) in  $t$  along the deterministic interpolant (i.e. holding  $\mathbf{z}$  and  $\boldsymbol{\varepsilon}$  fixed) gives the canonical *conditional velocity field* for this family,

$$u_t(\mathbf{x}_t | \mathbf{z}) = \dot{\alpha}_t \mathbf{z} + \dot{\beta}_t \boldsymbol{\varepsilon}, \quad (61)$$

which we may equivalently rewrite in  $\mathbf{x}$ -only form by eliminating  $\boldsymbol{\varepsilon} = (\mathbf{x}_t - \alpha_t \mathbf{z})/\beta_t$ :

$$u_t(\mathbf{x} | \mathbf{z}) = \left( \dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) \mathbf{z} + \frac{\dot{\beta}_t}{\beta_t} \mathbf{x}. \quad (62)$$

The conditional score and the conditional posterior mean (*a.k.a.* Tweedie’s formula) are equally explicit:

$$s_t(\mathbf{x} \mid \mathbf{z}) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x} \mid \mathbf{z}) = -\frac{\mathbf{x} - \alpha_t \mathbf{z}}{\beta_t^2}, \quad \mathbb{E}[\mathbf{z} \mid \mathbf{x}_t = \mathbf{x}] = \frac{\mathbf{x} - \beta_t \mathbb{E}[\varepsilon \mid \mathbf{x}_t = \mathbf{x}]}{\alpha_t}. \quad (63)$$

*Crucially*, all three quantities –  $u_t(\mathbf{x} \mid \mathbf{z})$ ,  $s_t(\mathbf{x} \mid \mathbf{z})$ , and the linear maps that recover  $\mathbf{z}$  from  $(\mathbf{x}, \varepsilon)$  or from  $(\mathbf{x}, u)$  – are closed-form once  $(\alpha_t, \beta_t, \dot{\alpha}_t, \dot{\beta}_t)$  are known. Conditional flow matching can therefore be implemented with no extra simulation cost beyond a single sample of  $\mathbf{z}$  and  $\varepsilon$  per training step.

A direct consequence of Theorem 5.4 and Eq. (61) is the following.

**Proposition 5.6** (Marginal velocity for Gaussian paths). For the Gaussian conditional path of Eq. (59), the marginal velocity field admits the equivalent representations

$$u_t(\mathbf{x}) = \mathbb{E}[\dot{\alpha}_t \mathbf{z} + \dot{\beta}_t \varepsilon \mid \mathbf{x}_t = \mathbf{x}] = \left( \dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) \mathbb{E}[\mathbf{z} \mid \mathbf{x}_t = \mathbf{x}] + \frac{\dot{\beta}_t}{\beta_t} \mathbf{x}, \quad (64)$$

i.e. it is an affine combination of the data-posterior mean  $\mathbb{E}[\mathbf{z} \mid \mathbf{x}_t]$  and  $\mathbf{x}_t$ , with coefficients fixed by the schedule.

This proposition makes it transparent that learning the velocity field is, up to a known affine map, the same as learning the data-posterior mean: a network that achieves  $\mathbf{u}_t^\theta(\mathbf{x}) \approx u_t(\mathbf{x})$  implicitly performs MMSE denoising of  $\mathbf{x}_t$  at every  $t$ , which is the standard interpretation of denoising score matching [26, 34].

### 5.2.2 Linear schedule

We adopt the *linear schedule*

$$\boxed{\alpha_t = t, \quad \beta_t = 1 - t, \quad \dot{\alpha}_t = 1, \quad \dot{\beta}_t = -1,} \quad (65)$$

which is the canonical rectified-flow / linear-interpolant choice of [10, 32, 33]. Substituting into Eq. (61) yields the strikingly simple form of the conditional velocity target,

$$u_t(\mathbf{x}_t \mid \mathbf{z}) = \mathbf{z} - \varepsilon, \quad \mathbf{x}_t = t \mathbf{z} + (1 - t) \varepsilon, \quad (66)$$

i.e. a *t-independent* vector pointing from the noise sample  $\varepsilon$  to the data sample  $\mathbf{z}$ . Geometrically, the conditional flow associated with the linear schedule is the segment  $t \mapsto t \mathbf{z} + (1 - t) \varepsilon$ , traversed at constant velocity  $\mathbf{z} - \varepsilon$ . The marginal flow that the network learns is the optimal-transport-like quantity that interpolates the two distributions in expectation [32].

The linear schedule has three properties that make it the natural default for direct flow matching on standardised data.

1. *t-independent regression target.* The right-hand side of Eq. (66) does not depend on  $t$  explicitly (the  $t$ -dependence enters only through  $\mathbf{x}_t$ ). The CFM loss therefore has homogeneous per-time-bin variance and uniform sampling  $t \sim \mathcal{U}[0, 1]$  is unbiased with respect to all multipoles of the patch power spectrum – no per- $t$  reweighting is required, which simplifies the optimisation.
2. *Bounded variance throughout the path.* The marginal variance is  $\text{Var}(\mathbf{x}_t \mid \mathbf{z}) = \beta_t^2 = (1 - t)^2$  at any  $t$ , with a controlled departure from the variance-preserving condition:  $\alpha_t^2 + \beta_t^2 = 1 - 2t(1 - t) \in [\frac{1}{2}, 1]$ . After the per-channel standardisation of Sec. 4.4 the data has unit variance per channel, so  $\text{Var}(\mathbf{x}_t) \leq 1$  at every  $t$  and the network never sees activations beyond the dynamic range it was initialised for.

3. *Endpoint singularity is harmless.* The factor  $1/\beta_t$  in the conditional score and in the alternative form of  $u_t$  in Eq. (62) is finite for any  $t < 1$ , and at sampling time we integrate strictly inside the open interval  $(0, 1)$  and analytically close the residual gap (Sec. 5.6).

A trigonometric schedule  $\alpha_t = \sin(\pi t/2)$ ,  $\beta_t = \cos(\pi t/2)$  – which is exactly variance-preserving,  $\alpha_t^2 + \beta_t^2 = 1$ , and aligns with the canonical variance-preserving DDPM family of [8, 9] – and a bounded variant that bounds  $\beta_t, \alpha_t \geq 0.02$  are also implemented as alternative regularisers; we have verified that they reach the same spectral validation score as the linear schedule in the present setting and report only the linear-schedule production runs below.

### 5.2.3 Implications for training and sampling

Equations (62)–(63) collectively imply that, for the Gaussian conditional path with the linear schedule, the training pipeline reduces to a single line of code – sample  $(\mathbf{z}, \varepsilon, t)$ , build  $\mathbf{x}_t = t\mathbf{z} + (1 - t)\varepsilon$ , regress  $\mathbf{u}_t^\theta(\mathbf{x}_t)$  towards  $\mathbf{z} - \varepsilon$  – and the sampling pipeline reduces to integrating  $d\mathbf{x}_t/dt = \mathbf{u}_t^\theta(\mathbf{x}_t)$  from  $\mathbf{x}_0 \sim p_{\text{init}}$ . Both operations are deterministic given the random draws made at training time; in particular, sampling has *no* extra randomness beyond the initial  $\mathbf{x}_0$ , which is the formal expression of the deterministic-flow choice of Sec. 5.1. The next subsections detail the parametric form of  $\mathbf{u}_t^\theta$ , the precise loss function used to train it, and the numerical integration of the resulting ODE.

## 5.3 Network architecture

The unknown marginal velocity field  $u_t : \mathbb{R}^D \times [0, 1] \rightarrow \mathbb{R}^D$  is parametrised by a deep neural network  $\mathbf{u}_t^\theta$  with parameters  $\theta$ . The architecture is a translation-equivariant U-Net [8, 25], adapted to single-channel temperature patches by  $D_4$  data augmentation (Sec. 5.4) and by a Karras-style [26] time-embedding pathway. The model is unconditional in the sense that the only side information presented to the network is the time variable  $t \in [0, 1]$ . Inputs and outputs share the shape  $(B, 1, 64, 64)$ , where  $B$  is the mini-batch size and the lone channel is the standardised  $T$  patch of Sec. 4.4. The upstream simulation pipeline still produces full  $(T, Q, U)$  maps; the present  $p(f)$  run trains and validates on the temperature sub-problem only.

### 5.3.1 Why a U-Net for CMB patches

The choice of a convolutional U-Net is motivated by three structural properties of the patch data.

1. *Approximate translation invariance.* On flat patches of angular extent  $L \sim 7^\circ$ , the gnomonic projection error is  $\mathcal{O}((L/\text{rad})^2) \sim 10^{-2}$  (Sec. 3.6), so the patch statistics inherit the translation invariance of the underlying spherically-isotropic field to better than one percent. Convolutional architectures realise this symmetry exactly.
2. *Multi-scale structure of the power spectrum.* The patch power spectrum spans approximately four decades between the largest patch mode and the patch Nyquist multipole, with several physical features (acoustic peaks in  $TT$ , the polarisation peak structure in  $EE$ , the lensing-induced  $BB$  tail) at well-separated angular scales. The pyramidal encoder–decoder topology of a U-Net with skip connections is natively well-suited to producing a velocity field that simultaneously captures all of these scales.
3. *Rotational/reflection equivariance.* On the small gnomonic patches used here, the scalar temperature field is approximately  $D_4$ -equivariant: random rotations and reflections are valid symmetry augmentations with no spin bookkeeping. This is applied at the data-augmentation level (Sec. 5.4) rather than baked into the layers.

### 5.3.2 Macro-structure

The U-Net has a five-stage encoder–bottleneck–decoder layout with channel schedule  $C = (32, 64, 128, 256)$  and two residual layers per stage:

1. A *stem* that lifts the single-channel input patch to  $C_0 = 32$  feature channels via a single  $3 \times 3$  convolution.
2. An *encoder* of three downsampling stages: at stage  $\ell$  the feature map is processed by two residual layers at width  $C_\ell$  and then halved in spatial extent by a strided  $3 \times 3$  convolution that simultaneously projects to width  $C_{\ell+1}$ . The bottleneck feature map is  $(B, 256, 8, 8)$ .
3. A *midcoder* of two residual layers at the bottleneck width  $C_3 = 256$ , which globally mixes the 64 bottleneck pixels.
4. A *decoder* that mirrors the encoder: at each stage the bottleneck output is added to the corresponding encoder skip (additive skips, not concatenations), upsampled by a factor of two by bilinear interpolation, and refined by two residual layers at the next-lower width.
5. An *output head* that projects the final  $C_0$ -channel feature map back to a single-channel output, producing the predicted velocity field  $\mathbf{u}_t^\theta(\mathbf{x})$ .

### 5.3.3 Time conditioning

The time variable enters through a learnable Fourier feature embedding [26]: a fixed-dimension vector

$$\text{Embed}(t) = \sqrt{2}[\sin(2\pi t \mathbf{w}), \cos(2\pi t \mathbf{w})] \in \mathbb{R}^{d_t}, \quad d_t = 64, \quad (67)$$

where the frequencies  $\mathbf{w} \in \mathbb{R}^{d_t/2}$  are trainable parameters initialised as  $\mathbf{w} \sim \mathcal{N}(0, I)$ . The embedding is then mapped, by a small two-layer perceptron with one hidden layer, to a per-channel bias  $\mathbf{h}_\ell(t) \in \mathbb{R}^{C_\ell}$  at every stage  $\ell$ , and broadcast over the spatial dimensions before being added to the post-convolution feature map of every residual layer. This is a FiLM-style conditioning [28] restricted to its additive component. Each residual layer has the form

$$\mathbf{x} \mapsto \mathbf{x} + \text{Block}_2(\text{Block}_1(\mathbf{x}) + \mathbf{h}_\ell(t)), \quad \text{Block}_i = \text{Conv}_{3 \times 3} \circ \text{GroupNorm} \circ \text{SiLU}, \quad (68)$$

with GroupNorm [29] to avoid batch-statistic instabilities and SiLU activations for smooth gradients. We do not use self-attention layers: at  $64 \times 64$  resolution the receptive field of the bottleneck already covers the full patch, and we verified empirically that bottleneck attention did not measurably improve the spectral diagnostics of Sec. 5.7 while approximately doubling the per-step training cost.

### 5.3.4 Parameter count

With the channel schedule  $C = (32, 64, 128, 256)$  and two residual layers per stage, the network has  $\sim 3.5$  MiB of float-32 parameters. The patch resolution is moderate ( $64 \times 64$ ), the model is unconditional, and the per-step compute is dominated by the convolutional backbone rather than the time-conditioning MLP, so training easily fits in  $\lesssim 4$  GB of GPU memory at the production batch size  $B = 250$  used below.

## 5.4 Training: pure CFM loss and optimisation

### 5.4.1 Loss function

The production  $p(f)$  run uses *pure* conditional flow matching: the network  $\mathbf{u}_t^\theta$  is trained as a direct vector-field predictor by minimising only the CFM mean-squared error of Eq. (55). No auxiliary regularisers enter the objective. At each minibatch step we draw a Monte-Carlo

sample  $(\mathbf{z}, \varepsilon, t)$  with  $\mathbf{z} \sim p_{\text{data}}$  (a clean  $T$  patch from the training split),  $\varepsilon \sim \mathcal{N}(0, I_D)$ , and  $t$  sampled on  $[t_{\min}, t_{\max}]$  as described below; we then form the linear-path state  $\mathbf{x}_t = \alpha_t \mathbf{z} + \beta_t \varepsilon$  and regress the predicted velocity against the closed-form conditional target  $\dot{\alpha}_t \mathbf{z} + \dot{\beta}_t \varepsilon$ . The training objective is

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{CFM}}(\boldsymbol{\theta}) = \mathbb{E}_{t, \mathbf{z}, \varepsilon} \left[ \left\| \mathbf{u}_t^{\boldsymbol{\theta}}(\mathbf{x}_t) - (\dot{\alpha}_t \mathbf{z} + \dot{\beta}_t \varepsilon) \right\|_2^2 \right]. \quad (69)$$

For the linear schedule of Eq. (65) this reduces to the equivalent form

$$\mathcal{L}_{\text{CFM}}(\boldsymbol{\theta}) = \mathbb{E}_{t, \mathbf{z}, \varepsilon} \left[ \left\| \mathbf{u}_t^{\boldsymbol{\theta}}(\mathbf{x}_t) - (\mathbf{z} - \varepsilon) \right\|_2^2 \right]. \quad (70)$$

By Proposition 5.5 this is the canonical, simulation-free flow-matching objective: minimising it makes  $\mathbf{u}_t^{\boldsymbol{\theta}}(\mathbf{x})$  converge to the marginal velocity  $u_t(\mathbf{x})$  of the path, which by Eq. (49) drives the deterministic ODE integrated at sampling time. The implementation in `FlowMatchingTrainer.get_train_loss` (cell 53) is exactly this single regression step.

**Relation to earlier, over-regularised runs.** Earlier notebook iterations augmented Eq. (70) with an endpoint-reconstruction (“ $x_0$ ”) anchor and a log-power radial Fourier penalty on patch spectra. Those terms are no longer used: they were found to overpin low- $\ell$  amplitude well before the CFM optimum was reached, after which the validation spectral score drifted upward even as the training loss kept falling. The present pure-CFM setup instead relies on EMA evaluation plus best-validation-spectral checkpointing (Sec. 5.5) to select a snapshot whose generated  $D_{\ell}^{TT}$  matches the held-out reference. For diagnostics and sampling we still map a noisy state  $(\mathbf{x}_t, t)$  to a clean-patch estimate via the linear posterior-mean projector

$$\hat{\mathbf{z}}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \frac{\dot{\beta}_t \mathbf{x}_t - \beta_t \mathbf{u}_t^{\boldsymbol{\theta}}(\mathbf{x}_t)}{\alpha_t \dot{\beta}_t - \dot{\alpha}_t \beta_t}, \quad (71)$$

but this map is *not* included in the training loss.

#### 5.4.2 Symmetry-aware data augmentation

Temperature is a scalar field on the patch. At each training step we apply a uniformly drawn element of the dihedral group  $D_4$ —a random  $k \times 90^\circ$  rotation followed by an optional reflection across the vertical axis—to the clean batch  $\{\mathbf{z}_i\}$  before constructing  $\mathbf{x}_t$ . No spin-2 sign bookkeeping is required. This is exact at the angular scale of our patches (where curvature corrections to spherical isotropy are subdominant; Sec. 3.6) and provides an exact  $8\times$  multiplication of the effective training-set size at zero memory cost.

#### 5.4.3 Time-sampling distribution

Training and sampling share the interval  $[t_{\min}, t_{\max}] = [0, 0.995]$ . The upper truncation avoids the  $1/(1-t)$  singularity in the conditional score and preserves a one-step margin from the boundary that matches the Heun integration grid of Sec. 5.6. Rather than drawing  $t$  uniformly, we use the power-law bias implemented in `_sample_train_t`: draw  $u \sim \mathcal{U}[0, 1]$ , set

$$\tau = \begin{cases} u & \text{if the bias power is 1,} \\ 1 - (1 - u)^p & \text{otherwise,} \end{cases} \quad t = t_{\min} + (t_{\max} - t_{\min}) \tau, \quad (72)$$

with production bias power  $p = 0.4$  (`train_t_bias_power=0.4`). Values  $p < 1$  slightly overweight low- $t$  regions of the path;  $p = 1$  recovers uniform sampling. Because the linear-path velocity target Eq. (66) is  $t$ -independent, any reasonable sampling density on  $[t_{\min}, t_{\max}]$  remains unbiased in expectation with respect to all multipoles; the mild  $p = 0.4$  choice is retained from the production checkpoint rather than being a strong inductive bias.

#### 5.4.4 Optimiser

We optimise  $\mathcal{L}(\theta)$  with AdamW [31] at base learning rate  $10^{-4}$  and weight decay  $10^{-4}$ , with the gradient norm clipped to unity before each update. The learning rate is held constant for the first 5,000 epochs and then cosine-annealed [30] to  $0.1 \times$  the base value over the remaining epochs. The mini-batch size is  $B = 250$  and the production budget is 10,000 epochs (Sec. 5.5 explains why this budget is paired with validation-spectral checkpointing rather than with early stopping). Training is launched from cell 58 with `run_spectral=True`, so the flat-sky  $TT$  score of Sec. 5.5.2 is evaluated in-loop every 500 epochs.

#### 5.4.5 Hyperparameter summary

Table 2 reports the production hyperparameters for the  $p(f)$  run documented here.

| Hyperparameter                    | Value                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------------|
| Prior / channels                  | $p(f)$ , single-channel $T$ only                                                      |
| Schedule                          | Linear: $\alpha_t = t$ , $\beta_t = 1 - t$                                            |
| $t$ -support                      | $[t_{\min}, t_{\max}] = [0, 0.995]$                                                   |
| $t$ -sampling bias power $p$      | 0.4                                                                                   |
| Sampling regime                   | deterministic flow, $\sigma = 0$                                                      |
| Training objective                | pure CFM MSE, Eq. (69)                                                                |
| Augmentation                      | $D_4$ on $T$ patches                                                                  |
| Optimiser                         | AdamW, weight decay $10^{-4}$                                                         |
| Base learning rate                | $10^{-4}$                                                                             |
| Gradient clip                     | unit Frobenius norm                                                                   |
| Mini-batch size                   | 250                                                                                   |
| Number of epochs                  | 10,000                                                                                |
| LR schedule                       | constant for the first 5,000 epochs, cosine-annealing to $0.1 \times$ base afterwards |
| EMA decay                         | $\rho = 0.99$                                                                         |
| EMA evaluation warmup             | 300 steps                                                                             |
| In-training spectral diagnostic   | $N_s = 32$ samples, $N_{\text{steps}} = 1600$ Heun substeps, every 500 epochs         |
| Validation-spectral checkpointing | best-spectral EMA snapshot kept as final answer (Sec. 5.5)                            |
| State clipping at sampling        | $ x  \leq 5$                                                                          |
| Output checkpoint                 | <code>checkpoints/unet_cmb_t_pure_cfm_linear_heun_ema_10.pt</code>                    |

Table 2: Production hyperparameters for the deterministic flow-matching  $p(f)$  prior. The foreground prior  $p(x)$  reuses the same `FlowMatchingTrainer` class and most of these settings; its training cell and objective are summarised separately in Sec. 7.1.

### 5.5 EMA averaging and validation-driven checkpointing

The bare optimiser trajectory  $\{\theta_n\}_{n \geq 0}$  produced by AdamW is, at its best, a stochastic walk around the local minimum of the training objective. Two practical issues follow: (i) the per-step weights have a non-negligible amount of optimisation noise that hurts evaluation; (ii) the training loss is only a noisy proxy for the quantity we ultimately care about, namely the agreement of the generated patches with the validation  $D_\ell^{TT}$ . We address both with a single mechanism: an exponential moving average (EMA) of the weights that is hot-swapped at validation time, plus a validation-time selection rule that keeps the best EMA snapshot across the entire run rather than the last one.

### 5.5.1 Exponential moving average of the weights

**Update rule.** At each optimiser step  $n$  we maintain a shadow copy  $\bar{\theta}_n \in \mathbb{R}^p$  of the model parameters, updated by

$$\bar{\theta}_{n+1} = \rho \bar{\theta}_n + (1 - \rho) \theta_{n+1}, \quad \rho \in (0, 1), \quad (73)$$

with  $\rho = 0.99$  in our runs (`ema_decay=0.99` passed to `FlowMatchingTrainer` in cell 58). Unrolling (73) from the initial shadow  $\bar{\theta}_0 = \theta_0$  gives the equivalent geometric-weight representation

$$\bar{\theta}_n = \rho^n \theta_0 + (1 - \rho) \sum_{k=1}^n \rho^{n-k} \theta_k, \quad (74)$$

i.e. a low-pass filter on the optimiser trajectory with effective averaging horizon

$$\tau_{\text{EMA}} = \frac{1}{1 - \rho}, \quad (75)$$

so for  $\rho = 0.99$  that is  $\tau_{\text{EMA}} \approx 100$  steps. Eq. (74) is a Polyak-style running average, and for any twice-differentiable loss with bounded Hessian near the optimum this estimator has strictly lower variance than the instantaneous iterate  $\theta_n$  at the same training token count.

**Implementation.** The EMA shadow only mirrors floating-point parameters of the model. Integer buffers (e.g. batch counters, batch-norm running stats with integer types) are excluded from (73) so that the multiplication by  $\rho$  is well-defined. Concretely, the `ModelEMA` helper in cell 45 of the notebook stores

$$\text{shadow}[k] = \rho \text{shadow}[k] + (1 - \rho) \theta[k] \quad \text{for every floating-point key } k, \quad (76)$$

which is the in-place form of (73). The `update` method is called once per optimiser step from inside the trainer’s training loop (cell 53). The same helper exposes the methods `store`, `copy_to`, `restore`: this triplet implements the hot-swap evaluation pattern described next.

**Hot-swap evaluation.** At each diagnostic step we evaluate the EMA weights, not the live weights, by the canonical `store/copy/restore` protocol:

1. `store`: snapshot the live floating-point parameters  $\theta_n$  into a backup buffer.
2. `copy_to`: overwrite the live parameters with the EMA shadow  $\bar{\theta}_n$ .
3. Run the validation-time forward passes (vector-field MSE, spectral sampling, etc.).
4. `restore`: write the backup buffer back into the live parameters so that the next optimiser step continues on the original trajectory.

This block is implemented at lines 3260–3277 of the notebook (see the in-loop guard `idx >= self.ema_eval_warmup_steps`). The 300-step EMA warmup avoids using a poorly-conditioned shadow early in training, where the running average is still biased towards the random initialisation. After the warmup, the EMA copy is systematically smoother than the live weights at the same compute, and substantially reduces the variance of the spectral diagnostic of the next subsection, in line with the empirical findings of [26, 27].

### 5.5.2 The spectral selection score

The training-loss proxy  $\mathcal{L}_{\text{CFM}}$  is locally minimised by the optimal marginal velocity  $u_t^*$ , but locally the noise floor of the regression is set by the irreducible Gaussian-conditional MMSE  $(\dot{\alpha}_t\beta_t - \dot{\beta}_t\alpha_t)^2/(\alpha_t^2 + \beta_t^2)$ , which carries no preference for any particular  $D_\ell^{TT}$  shape. Two distinct snapshots can therefore have indistinguishable  $\mathcal{L}_{\text{CFM}}$  but visibly different generated  $D_\ell^{TT}$  curves. We therefore score each EMA snapshot by an end-to-end test that mirrors the inference pipeline as closely as possible.

Let  $\bar{\theta}$  be the EMA snapshot to be scored. We sample  $N_s$  patches by integrating the deterministic ODE

$$\mathbf{x}_0 \sim \mathcal{N}(0, I_{1 \times 64 \times 64}), \quad \frac{d\mathbf{x}_t}{dt} = \mathbf{u}_{\bar{\theta}}^{\bar{\theta}}(\mathbf{x}_t), \quad t \in [t_{\min}, t_{\max}], \quad (77)$$

with the same Heun substep grid used at inference (Sec. 5.6), reconstruct  $\hat{\mathbf{z}} = \hat{\mathbf{x}}_0$  from the endpoint  $\mathbf{x}_{t_{\max}}$  via the linear posterior-mean projector of Eq. (71), and apply the same Gaussian pixel-space smoothing kernel  $\sigma_{\text{pix}}$  that is applied to the validation patches. Denote the resulting smoothed patch ensemble by  $\{\hat{\mathbf{z}}_i\}_{i=1}^{N_s}$ .

**Two log-space discrepancies.** We then form two scalar discrepancies between the generated ensemble and the held-out validation ensemble  $\{\mathbf{z}_j\}_{j=1}^{N_{\text{ref}}}$ :

- (a) **Bandpower-shape discrepancy.** Using the flat-sky bandpower estimator  $\hat{P}_b$  of Sec. 5.7.2, compute the binned  $D_\ell^{TT}$  of the generated and reference ensembles,  $\hat{D}_b^{\text{gen}}$  and  $\hat{D}_b^{\text{ref}}$ , on the band  $\ell \leq \ell_{\text{max}} = 1200$ . Define

$$\mathcal{S}_{\text{shape}}(\bar{\theta}) = \sqrt{\frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} [\log \hat{D}_b^{\text{gen}}(\bar{\theta}) - \log \hat{D}_b^{\text{ref}}]^2}, \quad (78)$$

the RMS log-relative bandpower error, where  $\mathcal{B}$  collects the bandpowers with strictly positive  $\hat{D}_b$  on both sides. This is the term `spectral_shape_log_rmse` computed inside `run_spectral_diagnostics` (cell 53). Working in log-space matches the dynamic range of the patch power spectrum (about four decades from large-scale modes to the patch Nyquist) and means a 10% relative deficit at any single  $\ell$  contributes the same amount whether it sits at low or high  $\ell$ .

- (b) **Pixel-variance discrepancy.** Let  $\sigma_c^{\text{gen}}, \sigma_c^{\text{ref}}$  denote the per-channel pixel-level standard deviations on the generated and reference ensembles ( $c = 1$  for the temperature-only run;  $c = 1, 2, 3$  in the full  $TQU$  extension). Define

$$\mathcal{S}_{\text{var}}(\bar{\theta}) = \sqrt{\frac{1}{C} \sum_{c=1}^C [\log \sigma_c^{\text{gen}}(\bar{\theta}) - \log \sigma_c^{\text{ref}}]^2}. \quad (79)$$

This is the term `spectral_std_log_12` of cell 53. It pins the integrated patch-level variance, which is essentially  $\sum_b D_b/(2\pi)$  but evaluated directly in pixel space and therefore robust to small  $\ell$ -binning artefacts.

**The combined score.** The scalar used for checkpoint selection is the sum

$$\mathcal{S}(\bar{\theta}) = \mathcal{S}_{\text{shape}}(\bar{\theta}) + \mathcal{S}_{\text{var}}(\bar{\theta}). \quad (80)$$

Both summands are L2 norms in log-space and therefore unitless and roughly comparable in magnitude after  $D_\ell^{TT}$  is restricted to  $\ell \leq \ell_{\text{max}}$ . (80) is computed inside `run_spectral_diagnostics` as

```
spectral_selection_score = (
    spectral_shape_log_rmse + spectral_std_log_l2
)
```

A monotone decrease of  $\mathcal{S}$  during training is what we want to see; in the over-regularised run that motivated the present pure-CFM ablation,  $\mathcal{S}$  reached a global minimum near epoch  $\sim 1000$  and then drifted back up monotonically to  $\sim 2.5\times$  that value, as the auxiliary log-power loss continued to push gradient into low- $\ell$  once the CFM term had already saturated.

**Sampling configuration.** For the production  $p(f)$  run we use  $N_s = 32$  patches and  $N_{\text{steps}} = 1600$  Heun substeps in the spectral diagnostic (the constructor arguments `spectral_n_samples=32`, `spectral_n_steps=1600` of cell 58). The smoothing scale  $\sigma_{\text{pix}}$  matches the kernel applied at the pre-comparison step in `run_spectral_diagnostics` (cell 53); in the  $TT$  pipeline this is the same  $\sigma_{\text{pix}} = 1.2$  pixels we use for the post-training  $D_\ell^{TT}$  figure. The reference ensemble is the held-out validation split of the saved  $p(f)$  patches.

### 5.5.3 Best-validation-spectral checkpointing

**Selection rule.** A scalar `best_spectral_score` is initialised to `None` at the start of training. At every diagnostic step where the spectral score  $\mathcal{S}$  is finite we update

$$\mathcal{S}^* \leftarrow \min(\mathcal{S}^*, \mathcal{S}(\bar{\theta}_n)), \quad (81)$$

and whenever a new minimum is achieved we additionally snapshot the live and the EMA weights:

$$\text{best\_spectral\_model\_state\_dict} \leftarrow \theta_n, \quad \text{best\_spectral\_ema\_state\_dict} \leftarrow \bar{\theta}_n, \quad (82)$$

together with the epoch index `best_spectral_epoch`. This logic lives at lines 4116–4128 of cell 53.

**What the saved checkpoint contains.** At the end of training, cell 58 writes a single file with both the last-epoch weights and the best-spectral snapshot:

$$\mathcal{C} = \{\theta_{\text{last}}, \bar{\theta}_{\text{last}}, \theta_n^*, \bar{\theta}_n^*, \mathcal{S}^*, n^*, \text{schedule \& sampler config, training history}\}, \quad (83)$$

saved as `checkpoints/unet_cmb_t_pure_cfm_linear_heun_ema_10.pt`. In the production run documented here the best score  $\mathcal{S}^* \approx 0.042$  is reached at epoch  $n^* = 7500$ . The downstream evaluation cell prefers  $\bar{\theta}^*$  over  $\bar{\theta}_{\text{last}}$  whenever it is present (cell 59 uses the key `best_spectral_ema_state_dict` when available; cell 53 implements the same fallback in the post-training sampling block). The “final answer” of the run is therefore the EMA snapshot at the epoch at which  $\mathcal{S}$  was lowest, not the last EMA snapshot.

**Why the best-along-the-way and not early stopping.** The two protocols are operationally similar but conceptually different. Early stopping halts training when  $\mathcal{S}$  stops improving for a patience window and discards everything afterwards; the best-along-the-way protocol used here continues training to the full 10,000-epoch budget and only *retains* the best snapshot. The two coincide when  $\mathcal{S}$  is monotonically non-increasing and diverge whenever it is not. With a generic non-convex loss surface and a finite-batch estimator of  $\mathcal{S}$ , the latter is essentially never the case, and continuing to train past a local plateau lets the optimiser explore a different basin while keeping the previous best in cold storage. For our problem this matters: the best-spectral epoch is typically reached well before the AdamW-with-cosine-anneal trajectory has fully relaxed, and the loss landscape continues to evolve. The cost of the extra epochs is small (the bulk of the per-step cost is already spent on the U-Net forward pass) and the upside is non-trivial robustness against late-stage optimisation drift.

**Why  $\mathcal{S}$  rather than the validation  $\mathcal{L}_{\text{CFM}}$ .** The validation  $\mathcal{L}_{\text{CFM}}$  is dominated by mid- $t$  contributions where  $\langle \|u_t\|^2 \rangle$  is largest, and even an optimal model retains the irreducible MMSE there. Two snapshots with identical validation  $\mathcal{L}_{\text{CFM}}$  can therefore have visibly different  $\hat{D}_\ell^{TT}$ . The spectral score  $\mathcal{S}$  is computed end-to-end through the same solver and same flat-sky bandpower estimator as the post-training  $D_\ell^{TT}$  figure, so it is monotonically related to the user-facing deliverable. Selecting on  $\mathcal{S}$  is therefore the cheapest way to ensure that the saved checkpoint matches the reference ensemble in the same metric space in which the result is ultimately reported.

#### 5.5.4 End-to-end protocol summary

Concretely, at every diagnostic step  $n$  where the training-time spectral diagnostic fires:

1. *EMA hot-swap.* **store** the live  $\theta_n$ , then **copy\_to** the shadow  $\bar{\theta}_n$  ((73)).
2. *Reverse sampling.* Draw  $N_s = 32$  initial states  $\mathbf{x}_0 \sim \mathcal{N}(0, I)$  and integrate (77) from  $t_{\min} = 0$  to  $t_{\max} = 0.995$  with  $N_{\text{steps}} = 1600$  Heun substeps and state clip  $|\mathbf{x}_t| \leq 5$ .
3. *Endpoint denoising.* Reconstruct  $\hat{\mathbf{x}}_0$  from  $\mathbf{x}_{t_{\max}}$  using Eq. (71); the simulator returns  $\mathbf{x}_{t_{\max}}$  rather than clean data.
4. *Spectral comparison.* Smooth both generated and reference patches with  $\sigma_{\text{pix}}$ , compute  $\hat{D}_b^{\text{gen}}$  and  $\hat{D}_b^{\text{ref}}$ , and form  $\mathcal{S} = \mathcal{S}_{\text{shape}} + \mathcal{S}_{\text{var}}$  via (78)–(80).
5. *Best-snapshot update.* If  $\mathcal{S} < \mathcal{S}^*$ , update  $\mathcal{S}^* \leftarrow \mathcal{S}$  and snapshot  $(\theta_n^*, \bar{\theta}_n^*, n^*)$  via (81).
6. *EMA hot-restore.* **restore** the backup so the next optimiser step continues on the live trajectory.

At the end of the 10,000-epoch run the saved checkpoint  $\mathcal{C}$  contains both the last-epoch and the best-spectral snapshots, and the downstream sampler defaults to  $\bar{\theta}^*$ .

### 5.6 Sampling: deterministic ODE integration

Once  $\theta$  is trained, drawing samples from the learned prior reduces to numerically integrating the deterministic ODE

$$\mathbf{x}_0 \sim p_{\text{init}} = \mathcal{N}(0, I_D), \quad \frac{d\mathbf{x}_t}{dt} = \mathbf{u}_t^\theta(\mathbf{x}_t), \quad t \in [0, t_{\max}], \quad (84)$$

with  $t_{\max} = 0.995$  as in training. The endpoint  $\mathbf{x}_{t_{\max}}$  is then mapped to a clean-patch estimate  $\hat{\mathbf{z}}$  by the linear posterior-mean projector of Eq. (71). Because the deterministic flow has no in-sampler stochasticity, the entire procedure is a smooth, fully differentiable map from the base draw  $\mathbf{x}_0 \sim p_{\text{init}}$  to the final patch – a property that we will rely on for the downstream guided-posterior sampler of the component-separation pipeline.

#### 5.6.1 Numerical integrator: Heun’s second-order method

We use Heun’s second-order ODE method on a uniform grid of  $N_{\text{steps}} = 1200$  time steps. For step size  $h$  and current state  $(\mathbf{x}_t, t)$ , Heun’s method advances to  $t + h$  via

$$\mathbf{x}'_t = \mathbf{x}_t + h \mathbf{u}_t^\theta(\mathbf{x}_t) \quad \blacktriangleright \text{predictor (Euler step)} \quad (85)$$

$$\mathbf{x}_{t+h} = \mathbf{x}_t + \frac{h}{2} [\mathbf{u}_t^\theta(\mathbf{x}_t) + \mathbf{u}_{t+h}^\theta(\mathbf{x}'_t)] \quad \blacktriangleright \text{corrector (trapezoidal)}. \quad (86)$$

This achieves  $\mathcal{O}(h^3)$  local truncation error against the  $\mathcal{O}(h^2)$  of the plain Euler method, at the cost of two velocity-field evaluations per step. The choice over Euler is motivated by the spectral diagnostics of Sec. 5.7: a first-order Euler integrator at the same compute budget

produces a small but visible  $\mathcal{O}(1\%)$  amplitude bias on the patch power spectrum at intermediate  $\ell$ , consistent with the standard analysis of [26]. With Heun the residual integrator bias is comparable to the cosmic-variance scatter of our finite ensemble of generated patches, and is therefore invisible in the validation diagnostic.

### 5.6.2 Terminal-state reconstruction

The integration is stopped one step short of the boundary, at  $t_{\max} = 0.995$ , to avoid the  $1/(1-t)$  singularity that the conditional vector field of the linear path acquires at  $t = 1$ . The residual  $1 - t_{\max} = 0.005$  is closed analytically by the linear posterior-mean estimator

$$\hat{\mathbf{z}} = \frac{\dot{\beta}_{t_{\max}} \mathbf{x}_{t_{\max}} - \beta_{t_{\max}} \mathbf{u}_{t_{\max}}^{\theta}(\mathbf{x}_{t_{\max}})}{\alpha_{t_{\max}} \dot{\beta}_{t_{\max}} - \dot{\alpha}_{t_{\max}} \beta_{t_{\max}}} \stackrel{\text{linear}}{=} \mathbf{x}_{t_{\max}} + (1 - t_{\max}) \mathbf{u}_{t_{\max}}^{\theta}(\mathbf{x}_{t_{\max}}), \quad (87)$$

which is exact to first order in the residual. This is preferable to integrating the ODE all the way to  $t = 1$ , because it avoids polluting the final state with the  $\mathcal{O}((1-t)^{-1})$  amplification of any small per-step bias near the data end of the path.

### 5.6.3 Stability safeguards

Two minor numerical safeguards complete the sampler. (i) After every Heun step the state is element-wise clipped to  $|\mathbf{x}_t| \leq 5$  in standardised units. The fraction of clipped components in production runs is below  $10^{-3}$ , so the clip is a soft safety net against rare, unphysical excursions of the ODE rather than an active constraint. (ii) The base draw  $\mathbf{x}_0 \sim p_{\text{init}}$  uses a fixed seed so that the entire sampler is bit-reproducible given the trained checkpoint.

### 5.6.4 Production sampling configuration

For the figures shown below, we draw  $N_{\text{gen}} = 64$  samples with  $N_{\text{steps}} = 1200$ ,  $t \in [0, 0.995]$ ,  $\sigma = 0$  (deterministic flow),  $|\mathbf{x}| \leq 5$  clipping, and the best-spectral checkpoint of Sec. 5.4.

## 5.7 Diagnostics: validating the learned prior

The natural figure of merit for a learned CMB prior is the agreement of the second moments of the patch ensemble between the generated samples and an independent reference ensemble drawn from the same underlying physics. The notebook is currently restricted to the temperature/intensity sub-problem, so the diagnostic discussion below specialises to the  $TT$  channel; the polarisation  $EE/BB/TE$  counterparts are recovered in the same form once the polarisation pipeline is reintroduced. The same logic applies to the foreground prior  $p(x)$ , evaluated on its own held-out validation split.

### 5.7.1 Reference ensembles

We compare the generated samples against two independent reference ensembles, so that systematic biases of the trained network can be separated from systematic biases of the patch / flat-sky estimator pipeline.

1. *Validation patches.* The held-out validation split of the saved prior dataset (Sec. 4.4). These patches share the entire upstream pipeline – CAMB spectra (with curved-sky lensing for  $p(f)$ ), HEALPix pixelisation, beam, scalar pixel window, gnomonic projection, mask, per-channel standardisation – with the training set, but are drawn from full-sky realisations that are statistically independent from the training realisations because of the by-map split. They are therefore a faithful empirical realisation of the target distribution  $p_{\text{data}}$  that the network is asked to learn.

2. *Monte Carlo theory band.* An independent batch of clean-patch realisations is constructed from a freshly drawn ensemble of full-sky Gaussian CMB realisations, generated from the lensed CAMB  $TT$  spectrum at the same beam, pixel window, and  $N_{\text{side}}$  as the training set, and processed through the identical patch-extraction and flat-sky estimator pipeline. This MC ensemble does *not* pass through any neural network: the only difference between it and the validation ensemble is the absence of the lenspyx remap, which is irrelevant for  $TT$  at the percent level on  $\ell < 1200$ .

A feature of the generated curve that is also present in the MC theory band must be an artefact of the patch / flat-sky estimator pipeline itself; conversely, a feature that is present only in the generated band is a genuine bias of the trained prior.

### 5.7.2 Flat-sky $TT$ bandpower estimator

For a flat-sky patch of side  $L = N\Delta x$  with pixel resolution  $\Delta x$ , write its discrete Fourier transform

$$\tilde{T}(\mathbf{k}) = \sum_{\mathbf{x}} T(\mathbf{x}) \exp(-i\mathbf{k}\cdot\mathbf{x}) (\Delta x)^2, \quad \mathbf{k} \in \left\{ \frac{2\pi}{L}(n_x, n_y) : n_{x,y} = -\frac{N}{2}, \dots, \frac{N}{2} - 1 \right\}. \quad (88)$$

The flat-sky multipole assigned to each Fourier pixel is

$$\ell(\mathbf{k}) = 2\pi |\mathbf{k}|, \quad (89)$$

which is the leading-order curved-to-flat-sky correspondence valid on the patch sizes we use (Sec. 3.6). Given a batch  $\{T_i\}_{i=1}^B$  of patches and a partition of the multipole axis into bandpowers  $\{\mathcal{B}_b\}$ , the isotropic two-point estimator at bin  $b$  is

$$\hat{P}_b = \frac{(\Delta x)^2}{N^2} \frac{1}{N_{\text{modes}}^{(b)}} \sum_{\mathbf{k}: \ell(\mathbf{k}) \in \mathcal{B}_b} \frac{1}{B} \sum_{i=1}^B |\tilde{T}_i(\mathbf{k})|^2, \quad (90)$$

with  $N_{\text{modes}}^{(b)} = |\{\mathbf{k} : \ell(\mathbf{k}) \in \mathcal{B}_b\}|$  the number of Fourier pixels in bandpower  $b$ . The factor  $(\Delta x)^2/N^2$  is the standard discrete-Fourier normalisation that makes  $\hat{P}_b$  a consistent estimator of the continuum patch power  $C_\ell^{TT}$  on the central band of the Fourier grid [14]. We additionally plot the CMB-style power

$$\hat{D}_b = \frac{\ell_b(\ell_b + 1)}{2\pi} \hat{P}_b, \quad \ell_b \equiv \text{bin centre}, \quad (91)$$

which is the conventional axis on which the  $TT$  acoustic peaks are most easily read off.

The number of Fourier pixels falling in an annulus  $\mathcal{B}_b = [\ell_b^L, \ell_b^R)$  is

$$N_{\text{modes}}^{(b)} = \frac{\pi((\ell_b^R)^2 - (\ell_b^L)^2)}{(\Delta k)^2} = \frac{((\ell_b^R)^2 - (\ell_b^L)^2) L^2}{4\pi}, \quad (92)$$

where  $\Delta k = 2\pi/L$  is the Fourier-grid spacing. We use Eq. (92) heavily below; it controls the per-bin cosmic variance and the relative information content of different binning conventions.

### 5.7.3 Bandpower binning conventions used in the notebook

Two bandpower partitions of the multipole axis are used in parallel because they have very different statistical properties. Both estimators take the same input batch of patches and the same Eq. (90); they differ only in the choice of bin edges  $\{\ell_b^L, \ell_b^R\}$  and bin centres  $\ell_b$ .

**Linear bandpowers (`flat_sky_tt_spectrum`).** The default linear estimator places  $N_{\text{bin}} = \min(N/2, 30)$  bins of equal width on  $[0, \ell_{\text{max}}]$ , with  $\ell_{\text{max}} = \max_{\mathbf{k}} \ell(\mathbf{k})$  the maximum Fourier multipole on the diagonal of the patch grid. Bin centres are arithmetic midpoints,

$$\ell_b^L = b\Delta\ell, \quad \ell_b^R = (b+1)\Delta\ell, \quad \ell_b = \frac{1}{2}(\ell_b^L + \ell_b^R), \quad \Delta\ell = \frac{\ell_{\text{max}}}{N_{\text{bin}}}, \quad (93)$$

for  $b = 0, \dots, N_{\text{bin}}-1$ , with the right edge of the last bin closed. Inserting Eq. (93) into Eq. (92) gives, at fixed  $\Delta\ell$ ,

$$N_{\text{modes}}^{(b), \text{lin}} = \frac{(2\ell_b) \Delta\ell L^2}{4\pi} \propto \ell_b \Delta\ell, \quad (94)$$

so the mode count per bin grows linearly with the bin centre. In particular, the lowest bin contains few modes (only those Fourier pixels at  $\ell \leq \Delta\ell$ , including the patch fundamental at  $\ell_{\text{fund}} = 2\pi/L$ ), while the highest bin aggregates many.

**Logarithmic bandpowers (`flat_sky_tt_spectrum_binned`).** The log-binned estimator places  $N_{\text{bin}} = 48$  bins of equal logarithmic width on  $[\ell_{\text{min}}, \ell_{\text{max}}]$ , with  $\ell_{\text{min}} = \min_{\mathbf{k}: \mathbf{k} \neq 0} \ell(\mathbf{k}) = \ell_{\text{fund}}$  and bin centres set to the geometric midpoint,

$$\ell_b^L = \ell_{\text{min}} e^{b \Delta \ln \ell}, \quad \ell_b^R = \ell_{\text{min}} e^{(b+1) \Delta \ln \ell}, \quad \ell_b = \sqrt{\ell_b^L \ell_b^R}, \quad \Delta \ln \ell = \frac{\ln \ell_{\text{max}} - \ln \ell_{\text{min}}}{N_{\text{bin}}}. \quad (95)$$

For each bin,  $\ell_b^R - \ell_b^L = \ell_b (e^{\Delta \ln \ell/2} - e^{-\Delta \ln \ell/2}) \approx \ell_b \Delta \ln \ell$ , and so Eq. (92) gives

$$N_{\text{modes}}^{(b), \text{log}} \approx \frac{\ell_b^2 \Delta \ln \ell L^2}{2\pi} \propto \ell_b^2, \quad (96)$$

so the mode count per bin grows quadratically with the bin centre. Empty bins (those for which  $N_{\text{modes}}^{(b)} = 0$  on the discrete grid) are dropped from the output. The log estimator is the convention adopted in most generative-model patch-spectrum figures and is therefore retained for stylistic compatibility, but it should be read with the cosmic-variance considerations of Sec. 5.7.7 in mind.

**Legacy single-input estimator (`flat_sky_tt_power_spectrum`).** For backward compatibility with the earliest notebook drafts, a single-input variant operating on  $(N, H, W)$  arrays of bare temperature patches is retained. It implements Eq. (90) with the linear binning of Eq. (93); the only difference from `flat_sky_tt_spectrum` is the input layout and a uniform-batch averaging convention. The two estimators give numerically identical results on  $(N, 1, H, W)$  inputs.

#### 5.7.4 Pixel-space Gaussian smoothing (`gaussian_smooth_patches_t`)

Both the reference and the generated patches are passed through a mild isotropic Gaussian filter before the spectrum is estimated. With  $G_\sigma$  the unit-area normalised 2D Gaussian of width  $\sigma_{\text{pix}}$  pixels,

$$G_\sigma(\mathbf{x}) = \frac{1}{2\pi\sigma_{\text{pix}}^2(\Delta x)^2} \exp\left(-\frac{|\mathbf{x}|^2}{2\sigma_{\text{pix}}^2(\Delta x)^2}\right), \quad (97)$$

the smoothed patch is the convolution

$$T_\sigma(\mathbf{x}) = (G_\sigma * T)(\mathbf{x}), \quad \tilde{T}_\sigma(\mathbf{k}) = \tilde{G}_\sigma(\mathbf{k}) \tilde{T}(\mathbf{k}), \quad (98)$$

with Fourier-space transfer function

$$\tilde{G}_\sigma(\mathbf{k}) = \exp\left(-\frac{1}{2}\sigma_{\text{pix}}^2(\Delta x)^2|\mathbf{k}|^2\right). \quad (99)$$

Equivalently, the smoothed bandpower estimator is

$$\hat{P}_b^\sigma = \frac{1}{N_{\text{modes}}^{(b)}} \sum_{\mathbf{k}: \ell(\mathbf{k}) \in \mathcal{B}_b} e^{-\sigma_{\text{pix}}^2 (\Delta x)^2 |\mathbf{k}|^2} \frac{1}{B} \sum_{i=1}^B |\tilde{T}_i(\mathbf{k})|^2 (\Delta x)^2 / N^2. \quad (100)$$

The kernel half-power scale is at  $\ell^* \approx 2\pi/(\sigma_{\text{pix}}\Delta x)$ . With the production setting  $\sigma_{\text{pix}} = 1.2$  at  $\Delta x = 6.7'$  this places  $\ell^* \approx 1.3 \times 10^3$ , deeply inside the band of validity of the small-patch flat-sky approximation. The smoothing acts as a high-pass suppression of artefacts of the deterministic ODE integrator and of pixel discretisation that contaminate the very-high- $\ell$  tail of the unsmoothed spectrum; because it is applied identically to the generated and reference batches, it does not bias the comparison and can be turned off by setting  $\sigma_{\text{pix}} = 0$ . In what follows,  $P$  refers to the unsmoothed power and  $P^\sigma$  to the smoothed power.

### 5.7.5 Integrated and band-limited variance diagnostics

In addition to the binned spectra, two scalar variance diagnostics are recorded from the very same patches. Let  $\sigma_{\text{tot}}^2$  denote the empirical patch variance, and  $\sigma_{\text{sm}}^2$  the variance of the same patches after the Gaussian smoothing of Sec. 5.7.4. By Parseval, the population versions satisfy

$$\sigma_{\text{tot}}^2 = \int \frac{d^2k}{(2\pi)^2} P(\mathbf{k}), \quad (101)$$

$$\sigma_{\text{sm}}^2 = \int \frac{d^2k}{(2\pi)^2} P(\mathbf{k}) e^{-\sigma_{\text{pix}}^2 (\Delta x)^2 |\mathbf{k}|^2}, \quad (102)$$

with empirical estimates obtained by direct pixel-domain  $\text{Var}(\cdot)$  on the (smoothed) patch arrays, which is what the notebook prints. Equation (101) is the integrated power, dominated by the high- $\ell$  tail (where most modes live); Eq. (102) suppresses high- $\ell$  exponentially and is therefore dominated by the low-to-medium  $\ell$  band, with effective cutoff  $\ell \lesssim \ell^*$ . The two scalar ratios

$$r_{\text{tot}} = \frac{\sigma_{\text{tot}}^{\text{gen}}}{\sigma_{\text{tot}}^{\text{ref}}}, \quad r_{\text{sm}} = \frac{\sigma_{\text{sm}}^{\text{gen}}}{\sigma_{\text{sm}}^{\text{ref}}}, \quad (103)$$

together provide a coarse but powerful test of whether any amplitude bias is uniform across angular scales. If the model is biased by an overall multiplicative factor on  $P$ , both ratios are equal. If  $r_{\text{sm}} < r_{\text{tot}}$ , the bias is concentrated at the larger angular scales weighted by Eq. (102); if  $r_{\text{sm}} > r_{\text{tot}}$ , the bias is concentrated at higher  $\ell$ . The expected “power-equivalent” under a uniform amplitude rescale, namely  $r_{\text{tot}}^2$ , is also reported as a sanity reference.

### 5.7.6 Variance-matched comparison

To isolate spectral shape mismatches from overall amplitude biases, we additionally compute a variance-matched generated batch

$$T_i^{\text{vm}}(\mathbf{x}) = \frac{\sigma_{\text{tot}}^{\text{ref}}}{\sigma_{\text{tot}}^{\text{gen}}} T_i^{\text{gen}}(\mathbf{x}), \quad (104)$$

with the rescaling factor estimated from the same batches used for the spectral comparison (this is the line `gen_vm_sp = gen_sp * _sc.reshape(1,1,1,1)` of `run_spectral_diagnostics`). Its bandpower follows immediately from linearity of Eq. (90),

$$\hat{P}_b^{\text{vm}} = \left( \frac{\sigma_{\text{tot}}^{\text{ref}}}{\sigma_{\text{tot}}^{\text{gen}}} \right)^2 \hat{P}_b^{\text{gen}}. \quad (105)$$

By construction the variance-matched and reference batches agree on  $\sigma_{\text{tot}}^2 = \int P d^2k / (2\pi)^2$ . Any residual disagreement of  $\hat{P}_b^{\text{vm}}$  with the reference  $\hat{P}_b^{\text{ref}}$  is therefore a pure shape bias: a

redistribution of power across  $\ell$  at fixed integrated variance, that no overall rescale can absorb. We also record the corresponding spectral mismatch metrics on both the raw and the variance-matched bandpowers, so that the contribution of amplitude and shape to the overall residual can be tracked independently across training.

### 5.7.7 Cosmic variance per bin

For a Gaussian random field, the bandpower estimator of Eq. (90) follows a chi-squared distribution and its variance is, to leading order in  $1/N_{\text{modes}}^{(b)}$  and  $1/B$ ,

$$\frac{\text{Var}(\hat{P}_b)}{P_b^2} \approx \frac{2}{B N_{\text{modes}}^{(b)}}, \quad (106)$$

where  $B$  is the number of patches and  $N_{\text{modes}}^{(b)}$  is given by Eq. (92). Three remarks follow.

1. *Linear vs. logarithmic binning at low  $\ell$ .* The same patch ensemble carries very different per-bin uncertainties under the two binnings of Sec. 5.7.3. At the patch fundamental  $\ell \approx \ell_{\text{fund}} \approx 50$  for our  $7^\circ$  patches, the linear estimator's first bin extends from  $\ell = 0$  to  $\ell = \Delta\ell \approx 76$  and aggregates roughly  $(\ell_{\text{fund}} + \Delta\ell)\Delta\ell L^2/(4\pi) \approx 7$  Fourier pixels. With  $B = 64$  patches this gives  $\sigma(\hat{P}_0)/P_0 \approx \sqrt{2/(64 \cdot 7)} \approx 6.7\%$ , a small enough scatter that any low- $\ell$  deficit of order  $\sim 10\%$  is statistically resolved at first-bin level. The log estimator, with  $\Delta \ln \ell \approx 0.083$ , places the same low- $\ell$  region across roughly  $\sim 10$  very narrow bins, each containing  $\lesssim 1$  Fourier pixel and a per-bin scatter of  $\sigma(\hat{P}_b)/P_b \approx \sqrt{2/64} \approx 18\%$ . The two visualisations therefore highlight different things: the linear plot exposes any low- $\ell$  amplitude deficit as a single high-precision data point, while the log plot resolves the same deficit across many independently-noisy points and is visually softened by the eye's tendency to trend-match.
2. *Visual interpretation.* A model bias smaller than the per-bin  $1\sigma$  scatter of Eq. (106) is statistically indistinguishable from a fluctuation. The Monte Carlo theory band of Sec. 5.6.1 provides a direct numerical estimate of this floor at every  $\ell$ , by replicating the per-bin scatter with independent realisations of the target distribution, and is the recommended visual ruler against which the generated curve is read.
3. *Practical recommendation.* For a strict go/no-go test on the  $TT$  amplitude at low  $\ell$ , the linear-binned plot is the relevant diagnostic. The log-binned plot is the appropriate visualisation when comparing across decades of  $\ell$  on a single axis, and should be read in conjunction with the per-bin uncertainty implied by Eq. (106).

### 5.7.8 Why this is the relevant test for a CMB prior

For a Gaussian field, the angular power spectrum is a sufficient statistic and an exact match of the second moments is mathematically equivalent to learning the distribution. For the lensed CMB the field is mildly non-Gaussian (the lensing-induced 4-point function leaves an imprint at the level of  $\sim 1\%$  of the variance at the Planck angular resolution), but on the angular scales resolved by the patch ( $\ell \lesssim 1200$ ) the second-moment test still captures the bulk of the information content. For the Galactic foregrounds the field is more strongly non-Gaussian, but the second-moment comparison remains the standard first-line diagnostic and is the quantity that downstream component-separation analyses are most sensitive to. We therefore treat the  $D_\ell^{TT}$  comparison, together with the integrated and band-limited variance diagnostics of Sec. 5.7.5 and the variance-matched comparison of Sec. 5.7.6, as the principal go/no-go tests of the trained prior, and reserve higher-order tests (cross-power spectrum residuals, pixel-domain histograms, and one-point statistics) for follow-up work.

### 5.7.9 Reading the diagnostics: amplitude vs. shape biases

Combining the binned spectra of Sec. 5.7.3, the variance ratios of Eq. (103), and the variance-matched comparison of Eq. (105) provides a sharp decomposition of any failure mode of the trained prior into two limiting cases.

- *Amplitude bias* (uniform multiplicative error on  $P$ ). Characterised by  $r_{\text{tot}} \approx r_{\text{sm}} \neq 1$  and a variance-matched curve  $\hat{P}_b^{\text{vm}}$  that closely tracks the reference at all  $\ell$ . Action items: train longer and/or rely on the validation-spectral checkpoint of Sec. 5.5 to pick a better EMA snapshot.
- *Shape bias* (scale-dependent error on  $P$ ). Characterised by  $r_{\text{sm}} \neq r_{\text{tot}}$  and by a variance-matched curve  $\hat{P}_b^{\text{vm}}$  that fails to track the reference in the same multipole range where  $\hat{P}_b^{\text{gen}}$  does. The sign of  $r_{\text{sm}} - r_{\text{tot}}$  localises the failure:  $r_{\text{sm}} < r_{\text{tot}}$  indicates a deficit at low  $\ell$ ,  $r_{\text{sm}} > r_{\text{tot}}$  at high  $\ell$ . Action items: increase model capacity or training budget; revisit the  $t$ -sampling bias; or add a targeted post-hoc spectral correction rather than reintroducing the auxiliary training losses abandoned in the pure-CFM setup.

The current state of the trained  $p(f)$  prior at the production run reported in Fig. 1 is most consistent with a shape bias localised at  $\ell \lesssim 200$ , with  $r_{\text{sm}} \approx 0.82$  noticeably smaller than  $r_{\text{tot}} \approx 0.92$ . The variance-matched curve closes most but not all of the residual at low  $\ell$ , confirming that the dominant failure mode is a redistribution of power at fixed integrated variance and is therefore a pure shape problem rather than a uniform amplitude undershoot.

## 6 From the trained vector field to the $D_\ell^{TT}$ plot

This section walks through, step by step, what the notebook does between “the network has finished training” and “the  $D_\ell^{TT}$  figure is on the screen”. Each subsection states the underlying mathematics and points to the exact construct in `dm_cmb.ipynb` that implements it. The central object is the estimator `flat_sky_tt_spectrum`, which converts a batch of generated temperature patches into a binned isotropic power spectrum  $\hat{P}_b$ ; the  $D_\ell^{TT}$  curve is a one-line cosmetic rescale of that estimator output.

The pipeline is the composition

$$\underbrace{\mathbf{x}_0 \sim \mathcal{N}(0, I)}_{\text{step 1}} \xrightarrow{\text{Heun ODE}} \underbrace{\mathbf{x}_{t_{\text{max}}}}_{\text{step 2}} \xrightarrow{\hat{\mathbf{x}}_0 = R(\mathbf{x}_t, t)} \underbrace{\hat{T}_i^{\text{norm}}}_{\text{step 3}} \xrightarrow{\mu + \sigma \cdot} \underbrace{T_i}_{\text{step 4}} \xrightarrow{G_{\sigma^*}} \underbrace{T_i^\sigma}_{\text{step 5}} \xrightarrow{\text{FFT+bin}} \underbrace{\hat{P}_b}_{\text{step 6}} \xrightarrow{\ell(\ell+1)/2\pi} \quad (107)$$

### 6.1 Step 1 – Initial noise in normalised patch units

A batch of  $B = N_{\text{GEN}}$  initial states is drawn from the simple base distribution

$$\mathbf{x}_0 \sim \mathcal{N}(0, I_{1 \times 64 \times 64}), \quad B = N_{\text{GEN}}, \quad (108)$$

exactly as the network was trained against (Sec. 5 of the main text). In the notebook this is the line

```
x0 = torch.randn(N_GEN, 1, 64, 64, device=device)
```

The single channel encodes temperature in the *normalised patch units* of the prior  $p(f)$ : the patches the network was trained on were standardised per-channel by  $\mu, \sigma$  stored in `prior.f`, so  $\mathbf{x}_0$  lives in that same standardised space.

## 6.2 Step 2 – Heun integration of the flow ODE

The trained network is the conditional vector field  $u^\theta : \mathbb{R}^{1 \times 64 \times 64} \times [0, 1] \rightarrow \mathbb{R}^{1 \times 64 \times 64}$ , which approximates the marginal velocity that transports  $p_{\text{init}} = \mathcal{N}(0, I)$  to the data distribution  $p_{\text{data}}$  along the Gaussian probability path

$$\mathbf{x}_t = \alpha_t \mathbf{x}_{\text{data}} + \beta_t \epsilon, \quad \alpha_t = 1 - t, \quad \beta_t = t, \quad t \in [0, 1]. \quad (109)$$

At inference time, samples are produced by integrating the deterministic flow ODE

$$\frac{d}{dt} \mathbf{x}_t = u^\theta(\mathbf{x}_t, t), \quad \mathbf{x}_{t_{\min}} = \mathbf{x}_0, \quad (110)$$

forward from  $t_{\min}=0.05$  to  $t_{\max}=0.995$  on a uniform grid of  $N_{\text{steps}}=1200$  points,

$$\{t_n\}_{n=0}^{N_{\text{steps}}-1} = t_{\min} + n h, \quad h = \frac{t_{\max} - t_{\min}}{N_{\text{steps}} - 1}. \quad (111)$$

This is set up by

```
sde          = trainer.build_sampling_sde()          # wraps unet as u^theta
simulator    = build_sampler(sde)                    # -> HeunSimulator (sigma=0)
ts_1d        = torch.linspace(SAMPLE_T_MIN, SAMPLE_T_MAX, N_STEPS, device=device)
ts           = ts_1d.view(1,-1,1,1,1).expand(N_GEN,-1,-1,-1,-1)
```

The Heun second-order step is implemented in `HeunSimulator.step` as the standard predictor–corrector pair

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + h u^\theta(\mathbf{x}_n, t_n) \quad (\text{Euler predictor}), \quad (112)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{2} \left[ u^\theta(\mathbf{x}_n, t_n) + u^\theta(\tilde{\mathbf{x}}_n, t_n + h) \right] \quad (\text{trapezoidal corrector}), \quad (113)$$

giving  $\mathcal{O}(h^3)$  local truncation error vs. the  $\mathcal{O}(h^2)$  of plain Euler. After each step, `Simulator.simulate` optionally clips the state to a hard box for numerical stability,

$$\mathbf{x}_{n+1} \leftarrow \text{clamp}(\mathbf{x}_{n+1}, -C, +C), \quad C = \text{STATE\_CLIP\_ABS} = 5, \quad (114)$$

which acts only on rare runaway pixels (the fraction of clipped pixels is printed by `clip_frac`). The full loop is

```
with torch.no_grad():
    xt_final = simulator.simulate(x0, ts, state_clip_abs=STATE_CLIP_ABS)
```

and returns the terminal state  $\mathbf{x}_{t_{\max}} \in \mathbb{R}^{B \times 1 \times 64 \times 64}$  in normalised patch units.

## 6.3 Step 3 – Endpoint denoising: $\mathbf{x}_{t_{\max}} \rightarrow \hat{\mathbf{x}}_0$

Because  $t_{\max} = 0.995 < 1$ , the terminal state is still slightly noisy:  $\mathbf{x}_{t_{\max}} = (1 - t_{\max}) \mathbf{x}_{\text{data}} + t_{\max} \epsilon$  in the exact-path picture of Eq. (109). The notebook therefore applies the closed-form endpoint reconstruction implied by the vector field and the linear schedule. For any path  $(\alpha_t, \beta_t)$  one has identically

$$u(\mathbf{x}_t, t) = \frac{\dot{\alpha}_t \beta_t - \alpha_t \dot{\beta}_t}{\beta_t} \mathbf{x}_{\text{data}} + \frac{\dot{\beta}_t}{\beta_t} \mathbf{x}_t, \quad (115)$$

which can be solved for  $\mathbf{x}_{\text{data}} = \hat{\mathbf{x}}_0$  when the determinant  $\det \equiv \alpha_t \dot{\beta}_t - \dot{\alpha}_t \beta_t$  is non-zero:

$$\hat{\mathbf{x}}_0(\mathbf{x}_t, t) = \frac{\dot{\beta}_t \mathbf{x}_t - \beta_t u^\theta(\mathbf{x}_t, t)}{\alpha_t \dot{\beta}_t - \dot{\alpha}_t \beta_t}. \quad (116)$$

For the linear schedule  $\alpha_t = 1 - t$ ,  $\beta_t = t$ , the determinant is  $\det = (1 - t) \cdot 1 - (-1) \cdot t = 1$ , so Eq. (116) collapses to

$$\hat{\mathbf{x}}_0(\mathbf{x}_t, t) = \mathbf{x}_t - t u^\theta(\mathbf{x}_t, t). \quad (117)$$

The implementation lives in `GaussianConditionalProbabilityPath.reconstruct_x0_from_vector_field` and is called via the thin wrapper `trainer.reconstruct_x0`:

```
t_final = torch.full((N_GEN, 1, 1, 1), SAMPLE_T_MAX, device=device)
generated = trainer.reconstruct_x0(xt_final, t_final).cpu().numpy()
```

The output `generated` is  $\{\hat{\mathbf{x}}_0^{(i)}\}_{i=1}^B \in \mathbb{R}^{B \times 1 \times 64 \times 64}$  – a batch of denoised patches in *normalised patch units*.

## 6.4 Step 4 – De-standardisation to physical units

When the comparison is to be made in  $\mu\text{K}_{\text{CMB}}$ -based units (`COMPARE_IN_PHYSICAL_UNITS=True`), each pixel of each patch is mapped back through the channel-wise mean/std stored in the prior dictionary,

$$T_i(\mathbf{x}) = \mu + \sigma \hat{\mathbf{x}}_0^{(i)}(\mathbf{x}), \quad (\mu, \sigma) = (\text{prior\_f['mean']}, \text{prior\_f['std']}), \quad (118)$$

implemented in

```
def _denormalize_from_prior(arr, prior_dict):
    mean = np.asarray(prior_dict['mean'], dtype=np.float32)
    std = np.asarray(prior_dict['std'], dtype=np.float32)
    return (arr * std + mean).astype(np.float32)
generated_base = _denormalize_from_prior(generated, prior_f)
```

In the normalised-domain comparison mode the inverse standardisation is skipped and the spectrum is computed directly on  $\hat{\mathbf{x}}_0^{(i)}$ . In both modes the reference batch `ref_base` is fed through the *same* de-standardisation/identity branch, so the generated and the reference are always in the same units before the spectrum is taken.

## 6.5 Step 5 – Optional Gaussian pixel smoothing

Both batches are passed through a mild isotropic 2D Gaussian filter of width  $\sigma_{\text{pix}} = \text{SMOOTH\_SIGMA\_PIX} = 1.2$  pixels (production setting),

$$T_i^\sigma(\mathbf{x}) = (G_{\sigma_{\text{pix}}} * T_i)(\mathbf{x}), \quad \tilde{T}_i^\sigma(\mathbf{k}) = e^{-\frac{1}{2}\sigma_{\text{pix}}^2(\Delta x)^2|\mathbf{k}|^2} \tilde{T}_i(\mathbf{k}), \quad (119)$$

implemented as

```
generated_sp = gaussian_smooth_patches_t(generated_base, SMOOTH_SIGMA_PIX)
ref_sp = gaussian_smooth_patches_t(ref_base, SMOOTH_SIGMA_PIX)
```

(the routine `gaussian_smooth_patches_t` is a thin `scipy.ndimage.gaussian_filter` wrapper). Because the same kernel is applied to generated and reference, the comparison is unbiased; the smoothing serves only to suppress sub-pixel artefacts of the ODE integrator at the patch Nyquist scale (Sec. 5.7.4). Setting `SMOOTH_SIGMA_PIX=0` reproduces the unsmoothed spectrum.

## 6.6 Step 6 – The estimator `flat_sky_tt_spectrum`

This is the heart of the section. The call

```
ells_gen, spectra_gen = flat_sky_tt_spectrum(generated_sp, reso_arcmin=RESO_ARCMIN)
```

returns  $(\ell_b)_{b=0}^{N_{\text{bin}}-1}$  and  $\{TT : \hat{P}_b\}$ . Below we walk through the function body, equation by equation. Throughout this subsection  $T_i$  denotes the (possibly smoothed) input patch `T_patches[i]` of shape  $(H, W) = (64, 64)$  and  $B = N$  is the number of patches.

**(6a) Pixel geometry.** The pixel scale is converted to radians,

$$\Delta x = \text{reso\_rad} = \frac{\pi}{180} \cdot \frac{\text{reso\_arcmin}}{60}, \quad (120)$$

which for `RESO_ARCMIN=6.7` gives  $\Delta x \approx 1.949 \times 10^{-3}$  rad. The patch side length in radians is  $L = W \Delta x \approx 7.46^\circ$ , so the patches are small enough for the flat-sky approximation (Sec. 3.6).

`reso_rad = np.deg2rad(reso_arcmin / 60.0)`

**(6b) Fourier grid.** `np.fft.fftfreq(N,d= $\Delta x$ )` returns the cycles-per-radian grid

$$k_m = \begin{cases} \frac{m}{N \Delta x}, & 0 \leq m \leq \lfloor N/2 \rfloor, \\ \frac{m-N}{N \Delta x}, & \lfloor N/2 \rfloor < m < N, \end{cases} \quad m = 0, \dots, N-1, \quad (121)$$

i.e. frequencies measured in inverse radians (not angular frequencies). The multipole assigned to each Fourier pixel is then  $\ell(\mathbf{k}) = 2\pi|\mathbf{k}|$  (this is the standard flat-sky correspondence; the factor of  $2\pi$  converts from cycles/radian to radians<sup>-1</sup>):

$$k_x[m] = \text{fftfreq}(W, \Delta x)[m], \quad k_y[n] = \text{fftfreq}(H, \Delta x)[n], \quad \ell(\mathbf{k}) = 2\pi\sqrt{k_x^2 + k_y^2}. \quad (122)$$

```
kx = np.fft.fftfreq(W, d=reso_rad)
ky = np.fft.fftfreq(H, d=reso_rad)
KX, KY = np.meshgrid(kx, ky)
K_mag = np.sqrt(KX**2 + KY**2)
ell_map = 2 * np.pi * K_mag
```

The largest multipole on the grid is  $\ell_{\max} = 2\pi\sqrt{2}/(2\Delta x) = \pi\sqrt{2}/\Delta x$  (corner of the FFT grid) and the patch fundamental is  $\ell_{\text{fund}} = 2\pi/L = 2\pi/(W\Delta x)$ .

**(6c) Linear bandpower partition.** The multipole axis is divided into

$$N_{\text{bin}} = \min(H/2, 30) \quad (123)$$

linear bins of equal width  $\Delta\ell = \ell_{\max}/N_{\text{bin}}$ :

$$\ell_b^L = b \Delta\ell, \quad \ell_b^R = (b+1) \Delta\ell, \quad \ell_b = \frac{1}{2}(\ell_b^L + \ell_b^R), \quad b = 0, \dots, N_{\text{bin}} - 1. \quad (124)$$

```
ell_max = ell_map.max()
num_bins = min(H // 2, 30)
bin_edges = np.linspace(0, ell_max, num_bins + 1)
ell_centres = 0.5 * (bin_edges[:-1] + bin_edges[1:])
```

The mode counts  $N_{\text{modes}}^{(b)}$  implied by this binning are given by Eq. (92) of the main text.

**(6d) Discrete periodogram with continuum normalisation.** For patch  $i$ , the function computes the unitary 2D DFT

$$\tilde{T}_i[\mathbf{k}] = \sum_{\mathbf{x}} T_i(\mathbf{x}) \exp(-2\pi i (k_x m + k_y n) \Delta x), \quad (125)$$

where  $\mathbf{x} = (m, n) \Delta x$ , and then forms the patch periodogram with the DFT–continuum bridge factor

$$P^{(i)}(\mathbf{k}) = |\tilde{T}_i[\mathbf{k}]|^2 \cdot \underbrace{\frac{(\Delta x)^2}{HW}}_{\equiv \text{norm}}. \quad (126)$$

```

norm = reso_rad**2 / (H * W)          # = (Delta x)^2 / (H W)
T_fft = np.fft.fft2(T_patches[i])
P_TT = np.abs(T_fft) ** 2 * norm

```

The factor  $(\Delta x)^2/(HW)$  is the standard normalisation that makes  $P^{(i)}(\mathbf{k})$  a consistent estimator of the continuum patch power on the central band of the FFT grid:<sup>1</sup> the per-patch population power satisfies  $\langle P^{(i)}(\mathbf{k}) \rangle \rightarrow C_{\ell(\mathbf{k})}^{TT}$  as  $H, W \rightarrow \infty$  at fixed  $L = W\Delta x$ , in the same convention as the binned estimator of Eq. (90).

**(6e) Radial annular average.** For each bin  $b$ , the function takes the unweighted mean of the periodogram over Fourier pixels falling in  $\mathcal{B}_b = [\ell_b^L, \ell_b^R]$ :

$$\bar{P}_b^{(i)} \equiv \frac{1}{N_{\text{modes}}^{(b)}} \sum_{\mathbf{k}: \ell(\mathbf{k}) \in \mathcal{B}_b} P^{(i)}(\mathbf{k}), \quad N_{\text{modes}}^{(b)} = \#\{\mathbf{k} : \ell(\mathbf{k}) \in \mathcal{B}_b\}. \quad (127)$$

```

for b in range(num_bins):
    mask = (ell_map >= bin_edges[b]) & (ell_map < bin_edges[b + 1])
    if mask.any():
        accum_TT[b] += P_TT[mask].mean()          # = bar{P}_b^{(i)}
        if i == 0:
            counts[b] += 1

```

The auxiliary array `counts` records, at  $i = 0$ , whether bin  $b$  contains at least one Fourier pixel; bins with  $N_{\text{modes}}^{(b)} = 0$  are flagged as invalid and returned as NaN.

**(6f) Batch average.** After the loop over patches, the function divides by  $N = B$ ,

$$\hat{P}_b = \frac{1}{B} \sum_{i=1}^B \bar{P}_b^{(i)} = \frac{1}{B} \sum_{i=1}^B \frac{1}{N_{\text{modes}}^{(b)}} \sum_{\mathbf{k}: \ell(\mathbf{k}) \in \mathcal{B}_b} |\tilde{T}_i[\mathbf{k}]|^2 \frac{(\Delta x)^2}{HW}. \quad (128)$$

```

valid = counts > 0
arr_TT = np.full(num_bins, np.nan)
arr_TT[valid] = accum_TT[valid] / max(N, 1)    # divide by B
return ell_centres, {'TT': arr_TT}

```

Equation (128) is the same estimator as Eq. (90) of the main text, written in the order “radial mean first, batch mean second” rather than “batch mean first, radial mean second”. The two orderings are algebraically identical because each Fourier pixel carries equal weight in each bin and each patch carries equal weight in the batch.

**Return value.** The tuple  $(\text{ells\_gen}, \text{spectra\_gen}) = ((\ell_b)_b, \{TT : (\hat{P}_b)_b\})$  is a discrete sampling of the binned isotropic patch power spectrum. It is what the downstream plotting code consumes.

## 6.7 Step 7 – From $\hat{P}_b$ to $\hat{D}_b^{TT}$

The CMB-style display variable is

$$\hat{D}_b^{TT} = \frac{\ell_b(\ell_b + 1)}{2\pi} \hat{P}_b, \quad (129)$$

multiplied element-wise at the bin centres of Eq. (124). In the notebook this is literally

---

<sup>1</sup>With NumPy’s non-unitary DFT, Parseval reads  $\sum_{\mathbf{k}} |\tilde{T}[\mathbf{k}]|^2 = HW \sum_{\mathbf{x}} |T(\mathbf{x})|^2$ . The pixel-domain variance is then  $\sigma_T^2 = (\Delta x)^2 \sum_{\mathbf{x}} |T|^2 / L^2 = \sum_{\mathbf{k}} P^{(i)}(\mathbf{k}) / [(2\pi)^2 (\Delta k)^{-2}] \cdot (2\pi)^2$  – i.e. the standard  $\sigma_T^2 = \int P(\mathbf{k}) d^2 k / (2\pi)^2$  Parseval identity in the  $2\pi|\mathbf{k}| = \ell$  convention used here.

```

dl_fac_ref = ells_ref * (ells_ref + 1) / (2 * np.pi)
dl_fac_gen = ells_gen * (ells_gen + 1) / (2 * np.pi)
ref_vals   = dl_fac_ref * spectra_ref['TT']
gen_vals   = dl_fac_gen * spectra_gen['TT']
ax.plot(ells_ref, ref_vals, 'k-', label=reference_label)
ax.plot(ells_gen, gen_vals, 'C0--', label='generated')
ax.set_xlabel(r'$\ell$'); ax.set_ylabel(r'$D_{\ell}^{-1}$'); ax.set_xscale('log')

```

The same construction is applied to the optional Monte-Carlo theory band  $\hat{P}_b^{\text{th}}$ ,

$$\hat{D}_b^{TT, \text{th}} = \frac{\ell_b(\ell_b + 1)}{2\pi} \hat{P}_b^{\text{th}}, \quad (130)$$

and overlaid as the dotted grey curve in the comparison figure (Fig. 1).

## 6.8 End-to-end summary

Combining Eqs. (108)–(129), the quantity actually plotted on the vertical axis of the  $D_\ell^{TT}$  figure is

$$\hat{D}_b^{TT} = \frac{\ell_b(\ell_b + 1)}{2\pi} \frac{1}{B} \sum_{i=1}^B \frac{1}{N_{\text{modes}}^{(b)}} \sum_{\mathbf{k}: \ell(\mathbf{k}) \in \mathcal{B}_b} \left| \mathcal{F}[\mu + \sigma R(\Phi_{t_{\min} \rightarrow t_{\max}}^{u^\theta}(\mathbf{x}_0^{(i)}), t_{\max})](\mathbf{k}) \right|^2 \frac{(\Delta x)^2}{HW}, \quad (131)$$

where

- $\mathbf{x}_0^{(i)} \sim \mathcal{N}(0, I_{64 \times 64})$  is the  $i$ -th initial-noise patch (step 1, Eq. 108);
- $\Phi_{t_{\min} \rightarrow t_{\max}}^{u^\theta}$  is the Heun flow map of the learned vector field  $u^\theta$ , integrating Eq. (49) from  $t_{\min}$  to  $t_{\max}$  on the grid of Eq. (111), with optional state clipping (Eq. 114);
- $R(\mathbf{x}_t, t) = \mathbf{x}_t - t u^\theta(\mathbf{x}_t, t)$  is the linear-schedule endpoint reconstruction of Eq. (117);
- $\mu + \sigma \cdot$  is the per-channel de-standardisation of Eq. (118), present only in physical-units mode;
- $\mathcal{F}$  is the 2D DFT `np.fft.fft2`, with the multipole grid of Eq. (122);
- the inner double sum is the linearly-binned isotropic estimator of Eq. (128), i.e. the return value of `flat_sky_tt_spectrum`;
- the prefactor  $\ell_b(\ell_b + 1)/(2\pi)$  converts the patch power  $\hat{P}_b$  to the CMB-style  $\hat{D}_b^{TT}$  of Eq. (129).

The optional pixel-domain Gaussian smoothing of Eq. (119) multiplies the integrand of the inner sum by  $\exp[-\sigma_{\text{pix}}^2(\Delta x)^2|\mathbf{k}|^2]$  (cf. the smoothed bandpower of Eq. 100).

The reference curve  $\hat{D}_b^{\text{ref}}$  in the same figure is obtained by applying steps 4–7 to the validation patches  $\{T_i^{\text{ref}}\}$ , with steps 1–3 (the neural sampling) replaced by a direct draw from the saved  $p(f)$  prior. Because both branches share the identical de-standardisation, smoothing, FFT, and binning, any disagreement between  $\hat{D}_b^{\text{gen}}$  and  $\hat{D}_b^{\text{ref}}$  is attributable to the trained vector field  $u^\theta$  – which is the point of the diagnostic.

**Inputs.** At each diagnostic step, with EMA weights  $\bar{\theta}_n$  active, we generate  $N_s = 32$  patches  $\{\hat{\mathbf{z}}_i\}_{i=1}^{N_s}$  by Heun integration of the deterministic flow ODE plus the linear posterior-mean endpoint reconstruction. The reference is a fixed held-out validation batch  $\{\mathbf{z}_j\}_{j=1}^{N_{\text{ref}}}$ . Both ensembles are passed through the same Gaussian pixel smoothing kernel  $\sigma_{\text{pix}}$  before any spectrum is taken.

**Bandpower estimates and band selection.** We denote by  $\hat{D}_b^{\text{gen}}(\bar{\theta}_n)$  and  $\hat{D}_b^{\text{ref}}$  the binned flat-sky  $TT$  bandpowers

$$\hat{D}_b = \frac{\ell_b(\ell_b + 1)}{2\pi} \hat{P}_b, \quad (132)$$

returned by `flat_sky_tt_spectrum` on the generated and reference ensembles, restricted to multipoles  $\ell \leq \ell_{\text{max}} = 1200$ . Define the bin index set

$$\mathcal{M} = \{ b : \hat{D}_b^{\text{gen}}, \hat{D}_b^{\text{ref}} \text{ finite and } > 0 \}, \quad (133)$$

which excludes bandpowers below the floor of the estimator (typically the  $\ell$ -bin containing the patch fundamental).

**Bandpower-shape term.** The first summand is the root-mean-square *log-relative* discrepancy across  $\mathcal{M}$ ,

$$\mathcal{S}_{\text{shape}}(\bar{\theta}_n) = \sqrt{\frac{1}{|\mathcal{M}|} \sum_{b \in \mathcal{M}} [\log \hat{D}_b^{\text{gen}}(\bar{\theta}_n) - \log \hat{D}_b^{\text{ref}}]^2} \quad (134)$$

i.e.  $\mathcal{S}_{\text{shape}} = \sqrt{\text{MSE}_{\log D}}$ . This is the quantity stored as `spectral_shape_log_rmse` in `run_spectral_diagnostics` and is a band-by-band measure of relative shape mismatch in  $D_\ell^{TT}$ .

**Pixel-variance term.** The second summand pins the integrated patch-level variance,

$$\mathcal{S}_{\text{var}}(\bar{\theta}_n) = \sqrt{\frac{1}{C} \sum_{c=1}^C [\log \sigma_c^{\text{gen}}(\bar{\theta}_n) - \log \sigma_c^{\text{ref}}]^2} \quad (135)$$

where  $\sigma_c^{\text{gen}}, \sigma_c^{\text{ref}}$  are the per-channel pixel-level standard deviations on the generated and reference ensembles (here  $C = 1$ ). This is the `spectral_std_log_12` field in the diagnostic.

**Combined score.** The scalar used for checkpoint selection is the unweighted sum of the two terms,

$$\mathcal{S}(\bar{\theta}_n) = \mathcal{S}_{\text{shape}}(\bar{\theta}_n) + \mathcal{S}_{\text{var}}(\bar{\theta}_n). \quad (136)$$

Both summands are dimensionless  $L^2$  norms in log-space and so are roughly comparable in magnitude after  $D_\ell^{TT}$  is restricted to  $\ell \leq \ell_{\text{max}}$ . The minimum of  $\mathcal{S}$  across the run is recorded as  $\mathcal{S}^*$  together with the corresponding EMA snapshot  $\bar{\theta}^*$ , which is the checkpoint returned at inference time.

## 7 Foreground prior $p(x)$ : training and post-training diagnostics

So far the formalism of Sec. 5 has been instantiated for the CMB prior  $p(f)$ . The same machinery is then trained from scratch on the foreground prior  $p(x)$  (PySM3 thermal-dust + synchrotron at 143 GHz, projected and standardised exactly as for  $p(f)$  in Sec. 4.4). Concretely, the only changes between the two runs are the data sampler (`prior_x` instead of `prior_f`) and the on-disk checkpoint name; the U-Net architecture, the linear schedule, the conditional flow-matching loss (Eq. 70), the optimiser, the EMA, and the Heun ODE sampler are reused verbatim. Distinct objects `path_x`, `unet_x`, `trainer_x` are used so that the  $p(f)$  run is preserved in memory and on disk.

## 7.1 Training configuration

The training cell is `cell 66`; it instantiates a fresh `FlowMatchingTrainer` and calls `train()` with `run_spectral=False`. Foreground patches are dominated by non-Gaussian morphology (galactic-plane filaments, log-normal-like one-point PDF), so the in-training flat-sky  $TT$  score that drives best-validation-spectral checkpointing for  $p(f)$  (Sec. 5.5.2) is no longer sufficient by itself, and we postpone the spectral evaluation to a dedicated post-training step. The live diagnostic panel therefore collapses to four sub-plots: training loss, validation vector-field MSE (Eq. 70 on a fixed validation batch), the  $L_{\text{val},x_0}$  posterior-mean reconstruction (Eq. 71), and the validation MSE by timestep bin. All hyperparameters match the  $p(f)$  run of Table 2; in particular the training objective is the plain CFM mean-squared error

$$\mathcal{L}_{\text{CFM}}^{(x)}(\boldsymbol{\theta}) = \mathbb{E}_{t, z \sim p(x), \varepsilon \sim \mathcal{N}(0, I)} \left\| u_t^{\boldsymbol{\theta}_x}(\mathbf{x}_t) - (\dot{\alpha}_t z + \dot{\beta}_t \varepsilon) \right\|^2, \quad (137)$$

with  $\mathbf{x}_t = \alpha_t z + \beta_t \varepsilon$  and the linear schedule  $\alpha_t = 1 - t, \beta_t = t$ . The output checkpoint is `checkpoints/unet_fg_t_pure_cfm_linear_heun_ema.pt` and records, alongside the last-epoch weights and EMA shadow, the `prior:'foreground'` tag in its `training.config`.

## 7.2 Per- $t$ velocity-field diagnostic

The first post-training diagnostic, in `cell 68`, is the same single-curve check used for  $p(f)$ : it pits the trained vector field against the analytic CFM target on a fine  $t$ -grid, with the optimal linear estimator under matched second-order statistics as a lower bound. Two curves are plotted versus  $t \in [t_{\min}, t_{\max}]$ :

**Trained MSE.** At each  $t$  on the grid we draw  $N_{\text{batches}}$  independent batches  $\{(\mathbf{z}^{(b,m)}, \varepsilon^{(b,m)})\}$  of size  $N_{\text{per } t}$  from the held-out validation foreground patches and standard Gaussian noise, form  $\mathbf{x}_t^{(b,m)} = \alpha_t \mathbf{z}^{(b,m)} + \beta_t \varepsilon^{(b,m)}$ , and measure

$$\text{MSE}_{\text{trained}}^{(b)}(t) = \frac{1}{N_{\text{per } t} N_{\text{pix}}} \sum_{m=1}^{N_{\text{per } t}} \left\| \hat{u}_t^{\boldsymbol{\theta}_x}(\mathbf{x}_t^{(b,m)}) - (\dot{\alpha}_t \mathbf{z}^{(b,m)} + \dot{\beta}_t \varepsilon^{(b,m)}) \right\|^2. \quad (138)$$

The plot shows the mean over batches and a  $\pm 1\sigma$  band over the batch-to-batch spread, providing an explicit Monte-Carlo uncertainty band.

**Wiener (Gaussian-prior) MMSE floor.** A natural lower bound for  $\text{MSE}_{\text{trained}}(t)$  is the per-pixel MSE achievable by the optimal *linear* estimator of  $u_t$  given  $\mathbf{x}_t$  when  $\mathbf{z}$  is replaced by a Gaussian field with the same second-order statistics. Working in the 2D Fourier basis with the empirical patch power spectrum  $P(\mathbf{k})$ ,

$$P(\mathbf{k}) = \frac{1}{N_{\text{patches}}} \sum_{i=1}^{N_{\text{patches}}} \frac{|\tilde{z}^{(i)}(\mathbf{k})|^2}{N_{\text{pix}}}, \quad (139)$$

the 2D Wiener filter on each independent mode gives a closed-form MMSE

$$\text{MMSE}_{\text{W}}(t) = \frac{1}{N_{\text{pix}}} \sum_{\mathbf{k}} \frac{(\dot{\alpha}_t \beta_t - \dot{\beta}_t \alpha_t)^2 P(\mathbf{k})}{\alpha_t^2 P(\mathbf{k}) + \beta_t^2}. \quad (140)$$

The trained curve should track or sit slightly below the Wiener floor: gaps below it quantify the gain from exploiting non-Gaussian foreground structure that the linear estimator cannot capture, while sitting visibly above it at any  $t$  is a tell-tale of undertraining at that point of the probability path.

### 7.3 Post-training summary statistics: power spectrum and Minkowski functionals

Because  $p(x)$  is heavily non-Gaussian, two-point statistics alone are not sufficient to evaluate the generative model. We therefore reproduce the two-panel diagnostic of Heurtel-Depeiges *et al.* 2023 [18] (their Fig. C.2): a flat-sky angular power spectrum and the three Minkowski functionals of standardised excursion sets. Both diagnostics compare  $N_{\text{gen}}$  unconditionally generated patches  $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^{N_{\text{gen}}}$ , sampled by integrating the deterministic ODE driven by  $\hat{u}_t^{\theta_x}$  from  $\mathbf{x}_0 \sim \mathcal{N}(0, I)$  on the time grid  $[t_{\min}, t_{\max}]$  with the Heun integrator and applying the linear-schedule endpoint reconstruction (Eq. 117), against  $N_{\text{ref}}$  held-out validation patches  $\{\mathbf{x}^{(i)}\}_{i=1}^{N_{\text{ref}}}$  drawn directly from `prior_x['val_norm']`. The implementation is in `cell 70`.

#### 7.3.1 Flat-sky angular power spectrum

The bandpower estimator is exactly the one defined in Sec. 5.7.2,

$$\hat{C}_\ell^{TT, \text{patch}} = \frac{1}{N_{\text{modes}}(\ell)} \sum_{|\mathbf{k}| \in b_\ell} \frac{|\tilde{T}(\mathbf{k})|^2 (\Delta x)^2}{HW}, \quad \ell = 2\pi|\mathbf{k}|, \quad (141)$$

implemented by `flat_sky_tt_spectrum_binned` (`cell 52`) with the log-spaced binning of Eq. (95). A fixed isotropic Gaussian smoothing of width  $\sigma_{\text{pix}}$  is applied to both ensembles before the estimator is taken; the corresponding pixel-domain kernel is the one of Eq. (119) and the helper is `gaussian_smooth_patches_t`. We report the ensemble-mean  $D_\ell^{TT} = \ell(\ell+1)C_\ell^{TT}/2\pi$  for both ensembles, restricted to multipoles  $\ell \leq \ell_{\max} = 1200$  for plotting clarity.

A single scalar shape-and-amplitude mismatch is reported alongside the curve as the log-relative bandpower RMS,

$$\text{RMSE}_{\log D} = \sqrt{\frac{1}{|\mathcal{M}|} \sum_{b \in \mathcal{M}} [\log \hat{D}_b^{\text{gen}} - \log \hat{D}_b^{\text{ref}}]^2}, \quad (142)$$

with  $\mathcal{M}$  the set of bins where both estimators are finite and positive. Because  $\log(D^{\text{gen}}/D^{\text{ref}}) \approx (D^{\text{gen}} - D^{\text{ref}})/D^{\text{ref}}$  at small mismatch,  $100(\exp(\text{RMSE}_{\log D}) - 1)\%$  is interpretable approximately as a per-bin amplitude mismatch in percent.

#### 7.3.2 Minkowski functionals

The Minkowski functionals are the canonical morphological non-Gaussianity diagnostic for 2D random fields [19, 20]. For each patch  $x$ , we standardise  $\tilde{x} = (x - \langle x \rangle)/\text{std}(x)$  and define, for each threshold  $\nu \in \mathbb{R}$ , the excursion set

$$A_\nu(x) = \{ \mathbf{p} \in \Omega : \tilde{x}(\mathbf{p}) \geq \nu \}, \quad (143)$$

where  $\Omega$  is the patch domain ( $H \times W$  pixels). The three functionals are

$$V_0(\nu) = \frac{|A_\nu|}{|\Omega|}, \quad V_1(\nu) = \frac{\text{Per}(A_\nu)}{4|\Omega|}, \quad V_2(\nu) = \frac{\chi(A_\nu)}{2\pi|\Omega|}, \quad (144)$$

with  $|A_\nu|$  the area,  $\text{Per}$  the perimeter and  $\chi$  the Euler characteristic of the excursion set; the prefactors are the conventional 2D Hadwiger normalisations. We compute each functional patch-by-patch and report the mean and  $\pm 1\sigma$  band over the ensemble.

**Discrete estimators on a square pixel grid.** With pixel side  $\Delta x = 1$  in patch units and  $|\Omega| = N_{\text{pix}} = HW$ , the discrete estimators implemented by `minkowski_functionals` in cell 70 are:

- Excursion area:  $\hat{V}_0(\nu) = \frac{1}{N_{\text{pix}}} \sum_{\mathbf{p}} \mathbf{1}[\tilde{x}(\mathbf{p}) \geq \nu]$ , the fraction of pixels above threshold.
- Perimeter:  $\hat{V}_1(\nu) = \frac{1}{4N_{\text{pix}}} \# \{(\mathbf{p}, \mathbf{p}') : \mathbf{p} \sim_4 \mathbf{p}', \mathbf{1}[\tilde{x}(\mathbf{p}) \geq \nu] \neq \mathbf{1}[\tilde{x}(\mathbf{p}') \geq \nu]\}$ , the count of 4-connectivity edges that cross the level set, normalised by  $4|\Omega|$ .
- Euler characteristic:  $\hat{V}_2(\nu) = \frac{1}{2\pi N_{\text{pix}}} [\# \text{components}(A_\nu) - \# \text{holes}(A_\nu)]$ , with components extracted by 4-connectivity flood-fill (`scipy.ndimage.label`) and holes defined as connected components of the complement that do not touch the patch boundary.

Per-patch standardisation makes these estimators amplitude-independent and isolates morphological information: any rescaling  $x \mapsto \lambda x + c$  with  $\lambda > 0$  leaves  $\tilde{x}$  invariant, so the  $V_k$  depend only on the shape of the field. Departures of  $V_0, V_1$  from symmetry around  $\nu = 0$  – and of  $V_2$  from a perfect odd function of  $\nu$  – are direct signatures of the non-Gaussian morphology of the foreground field that the power spectrum (141) cannot detect.

**Reading the diagnostic.** A trained  $p(x)$  prior that captures both the second-order spectrum and the non-Gaussian morphology should yield (i)  $\hat{D}_\ell^{\text{gen}}$  tracking  $\hat{D}_\ell^{\text{ref}}$  with  $\text{RMSE}_{\log D}$  small (a few tens of percent or below per bin), and (ii) all three Minkowski functionals with mean curves and  $\pm 1\sigma$  bands overlapping the reference. Mismatches partition cleanly:

- Power spectrum off but Minkowski functionals match: a global amplitude/shape mismatch in second-order statistics. Most often indicates undertraining at high  $\ell$  (visible as a deficit at small angular scales).
- Power spectrum matches but Minkowski functionals deviate (especially asymmetry of  $V_0, V_1$  around  $\nu = 0$  or sign of  $V_2$  at moderate  $\nu$ ): the model has learned the second-order statistics but is generating closer-to-Gaussian fields than the reference. This is the typical failure mode of pure CFM on highly non-Gaussian foregrounds and motivates more elaborate priors (scattering transforms, conditional models) in subsequent work.

## References

- [1] M. Zaldarriaga and U. Seljak, “All-sky analysis of polarization in the microwave background,” *Physical Review D* **55** (1997), 1830–1840.
- [2] M. Kamionkowski, A. Kosowsky, and A. Stebbins, “Statistics of cosmic microwave background polarization,” *Physical Review D* **55** (1997), 7368–7388.
- [3] K. M. Górski, E. Hivon, A. J. Banday, B. D. Wandelt, F. K. Hansen, M. Reinecke, and M. Bartelmann, “HEALPix: A Framework for High-Resolution Discretization and Fast Analysis of Data Distributed on the Sphere,” *The Astrophysical Journal* **622** (2005), 759–771.
- [4] A. Lewis, A. Challinor, and A. Lasenby, “Efficient Computation of Cosmic Microwave Background Anisotropies in Closed Friedmann–Robertson–Walker Models,” *The Astrophysical Journal* **538** (2000), 473–476.
- [5] N. Aghanim et al. (Planck Collaboration), “Planck 2018 results. I. Overview and the cosmological legacy of Planck,” *Astronomy & Astrophysics* **641** (2020), A1.
- [6] B. Thorne, J. Dunkley, D. Alonso, and S. Næss, “The Python Sky Model: software for simulating the Galactic microwave sky,” *Monthly Notices of the Royal Astronomical Society* **469** (2017), 2821–2833.
- [7] A. Zonca, T. A. J. Louis, B. Thorne, N. Krachmalnicoff, and J. Borrill, “PySM 3: A Python Sky Model for simulation of Galactic microwave emission,” software/documentation release, 2021.
- [8] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems 33* (2020).
- [9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” in *International Conference on Learning Representations* (2021).
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in *International Conference on Learning Representations* (2023).
- [11] R. M. Sullivan, L. T. Hergt, and D. Scott, “Methods for CMB map analysis,” *arXiv:2410.12951* (2024), [doi:10.48550/arXiv.2410.12951](https://doi.org/10.48550/arXiv.2410.12951).
- [12] D. Scott and G. F. Smoot, “Cosmic Microwave Background,” in *Review of Particle Physics*, Particle Data Group collaboration, *Physical Review D* **110** (2024), 030001, pp. 524–534, [doi:10.1103/PhysRevD.110.030001](https://doi.org/10.1103/PhysRevD.110.030001).
- [13] A. Zonca, L. P. Singer, D. Lenz, M. Reinecke, C. Rosset, E. Hivon, and K. M. Górski, “healpy: equal area pixelization and spherical harmonics transforms for data on the sphere in Python,” *Journal of Open Source Software* **4** (2019), 1298, [doi:10.21105/joss.01298](https://doi.org/10.21105/joss.01298).
- [14] E. Hivon, K. M. Górski, C. B. Netterfield, B. P. Crill, S. Prunet, and F. Hansen, “MASTER of the Cosmic Microwave Background Anisotropy Power Spectrum: A Fast Method for Statistical Analysis of Large and Complex Cosmic Microwave Background Data Sets,” *The Astrophysical Journal* **567** (2002), 2–17.
- [15] D. Alonso, J. Sanchez, A. Slosar, and the LSST Dark Energy Science Collaboration, “A unified pseudo- $C_\ell$  framework: NaMaster,” Astrophysics Source Code Library, ascl:2307.048 (2023).

- [16] A. Challinor, P. Fosalba, D. Mortlock, M. Ashdown, B. Wandelt, and K. Górski, “All-sky convolution for polarimetry experiments,” *Physical Review D* **62** (2000), 123002.
- [17] U. Seljak and M. Zaldarriaga, “Signature of Gravity Waves in the Polarization of the Microwave Background,” *Physical Review Letters* **78** (1997), 2054–2057.
- [18] D. Heurtel-Depeiges, B. Burkhart, R. Ohana, and B. Régalo-Saint Blancard, “Removing Dust from CMB Observations with Diffusion Models,” in *Machine Learning and the Physical Sciences Workshop, NeurIPS* (2023), *arXiv:2310.16285*.
- [19] K. R. Mecke, T. Buchert, and H. Wagner, “Robust morphological measures for large-scale structure of the Universe,” *Astronomy & Astrophysics* **288** (1994), 697–704, *arXiv:astro-ph/9312028*.
- [20] J. Schmalzing and T. Buchert, “Beyond Genus Statistics: A Unifying Approach to the Morphology of Cosmic Structure,” *The Astrophysical Journal Letters* **482** (1997), L1–L4, *arXiv:astro-ph/9702130*.
- [21] M. Reinecke, S. Belkner, and J. Carron, “Improved cosmic microwave background (de-)lensing using a generalized non-uniform fast Fourier transform,” *Astronomy & Astrophysics* **678** (2023), A165, [doi:10.1051/0004-6361/202346717](https://doi.org/10.1051/0004-6361/202346717), *arXiv:2304.10431*. Software: [lenspyx](#) (wraps [DUCC](#)).
- [22] A. Lewis and A. Challinor, “Weak gravitational lensing of the CMB,” *Physics Reports* **429** (2006), 1–65, [doi:10.1016/j.physrep.2006.03.002](https://doi.org/10.1016/j.physrep.2006.03.002), *arXiv:astro-ph/0601594*.
- [23] J. Carron, M. Mirmelstein, and A. Lewis, “CMB lensing from Planck PR4 maps,” *Journal of Cosmology and Astroparticle Physics* **09** (2022), 039, [doi:10.1088/1475-7516/2022/09/039](https://doi.org/10.1088/1475-7516/2022/09/039), *arXiv:2206.07773*.
- [24] M. Millea, E. Anderes, and B. D. Wandelt, “Sampling-based inference of the primordial CMB and gravitational lensing,” *Physical Review D* **102** (2020), 123542, [doi:10.1103/PhysRevD.102.123542](https://doi.org/10.1103/PhysRevD.102.123542), *arXiv:2002.00965*.
- [25] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Lecture Notes in Computer Science **9351** (2015), 234–241, [doi:10.1007/978-3-319-24574-4\\_28](https://doi.org/10.1007/978-3-319-24574-4_28), *arXiv:1505.04597*.
- [26] T. Karras, M. Aittala, T. Aila, and S. Laine, “Elucidating the Design Space of Diffusion-Based Generative Models,” in *Advances in Neural Information Processing Systems* **35** (2022), 26565–26577, *arXiv:2206.00364*.
- [27] T. Salimans and J. Ho, “Progressive Distillation for Fast Sampling of Diffusion Models,” in *International Conference on Learning Representations* (2022), *arXiv:2202.00512*.
- [28] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual Reasoning with a General Conditioning Layer,” in *Proceedings of the AAAI Conference on Artificial Intelligence* **32** (2018), 3942–3951, *arXiv:1709.07871*.
- [29] Y. Wu and K. He, “Group Normalization,” in *Proceedings of the European Conference on Computer Vision (ECCV)* (2018), 3–19, *arXiv:1803.08494*.
- [30] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *International Conference on Learning Representations* (2017), *arXiv:1608.03983*.

- [31] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *International Conference on Learning Representations* (2019), *arXiv:1711.05101*.
- [32] X. Liu, C. Gong, and Q. Liu, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow,” *arXiv:2209.03003* (2022).
- [33] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden, “Stochastic Interpolants: A Unifying Framework for Flows and Diffusions,” *arXiv:2303.08797* (2023).
- [34] P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” *Neural Computation* **23**, 1661 (2011).
- [35] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” in *Advances in Neural Information Processing Systems* **32** (2019), *arXiv:1907.05600*.
- [36] H. Chung, J. Kim, M. T. McCann, M. L. Klasky, and J. C. Ye, “Diffusion Posterior Sampling for General Noisy Inverse Problems,” in *International Conference on Learning Representations* (2023), *arXiv:2209.14687*.
- [37] C. Saharia *et al.*, “Image Super-Resolution via Iterative Refinement,” *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**, 4713 (2023), *arXiv:2104.07636*.

Power Spectra (N\_gen=64, N\_ref=3277 val; ell<=1200.0; smooth=1.2px; steps=1200; clip=5.0; domain=uK\_CMB-based map units)

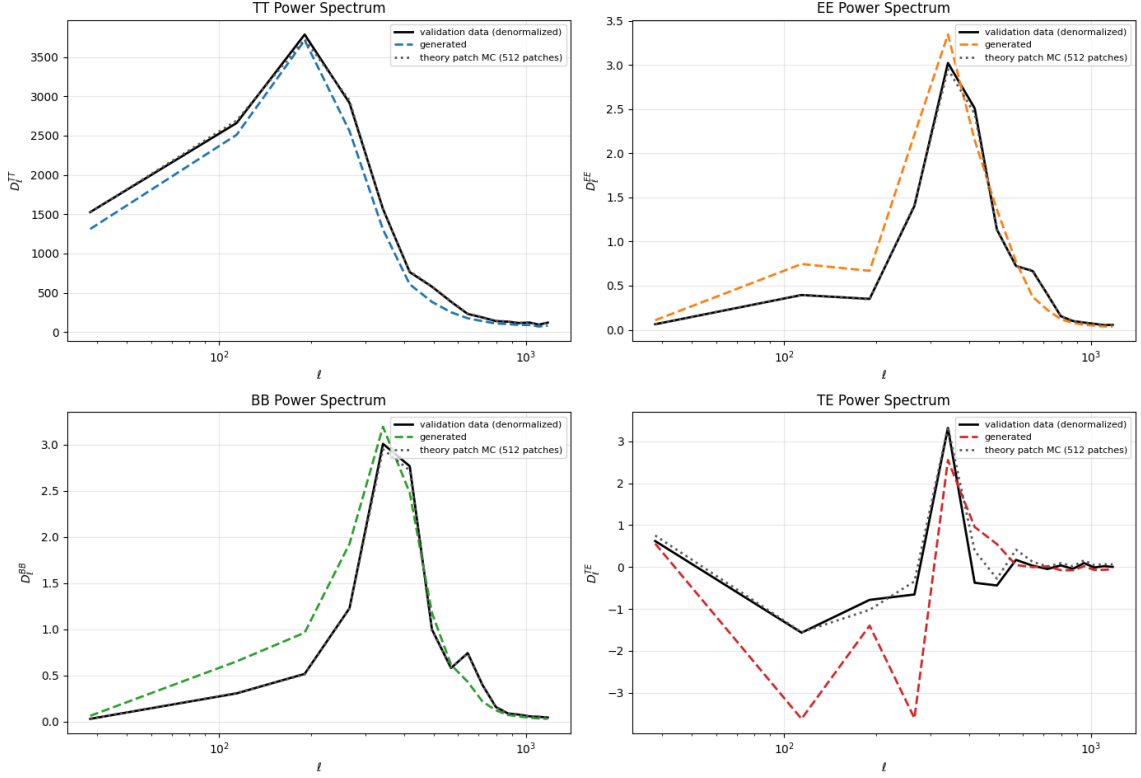


Figure 1: Post-training flat-sky  $D_\ell$  comparison for the trained  $p(f)$  prior in  $\mu\text{K}_{\text{CMB}}$ -based map units (after de-standardisation), shown here on the legacy four-panel layout produced by an earlier  $(T, Q, U)$ -channel run of the notebook for visual context. The current notebook is restricted to temperature, and the relevant panel is the top-left  $D_\ell^{TT}$ . Solid black: validation patches drawn from the saved  $p(f)$  prior, the empirical reference distribution. Dashed coloured: patches generated by the trained deterministic flow-matching model, sampled with the Heun second-order ODE integrator at  $N_{\text{steps}} = 1200$ ,  $t \in [0, 0.995]$ , and a state-clipping bound  $|\mathbf{x}| \leq 5$ . Dotted grey: Monte Carlo theory band built from independent Gaussian full-sky realisations of the lensed CAMB spectra passed through the identical patch and flat-sky pipeline. The agreement between the validation and theory bands is a calibration check on the patch / flat-sky estimator pipeline itself; the agreement between the generated and validation bands is the direct test of the trained prior.

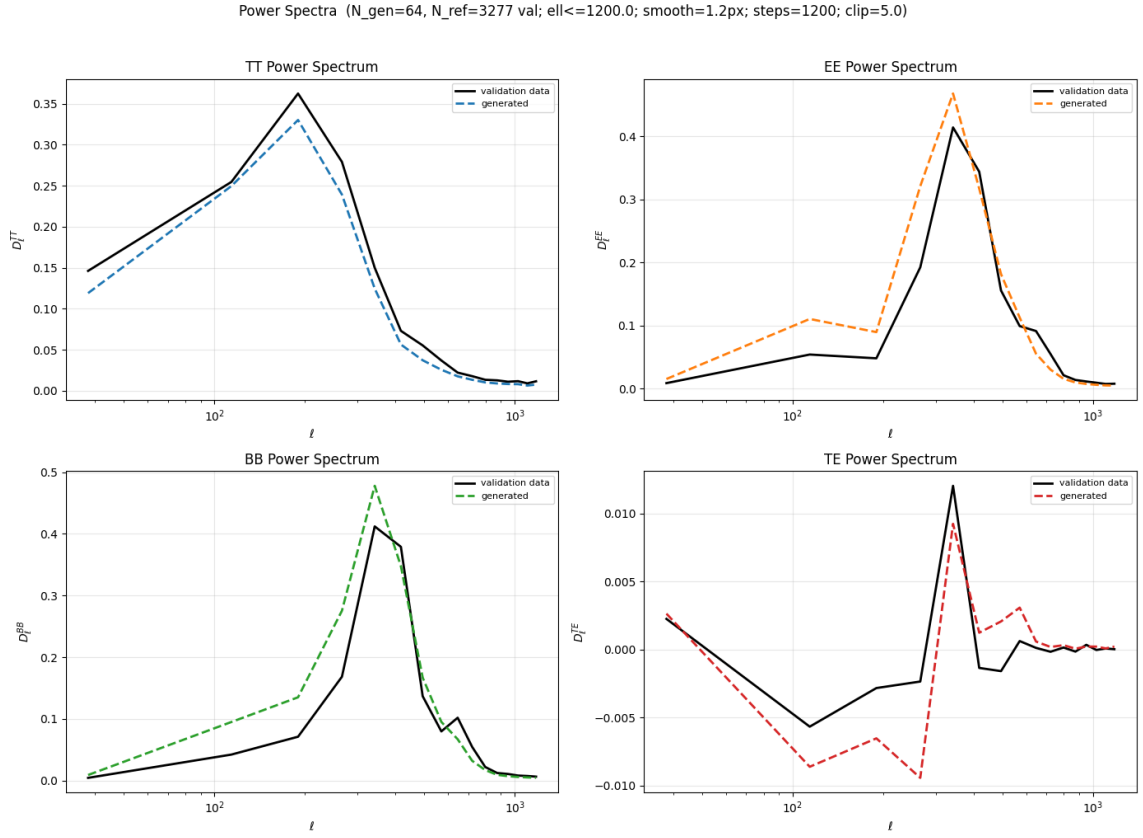


Figure 2: Same as Fig. 1 but in normalised patch units, i.e. before applying the inverse standardisation of Sec. 4.4. This is the domain in which the network is trained.